

Aristotle University of Thessaloniki
Department of Electrical and Computer Engineering
Telecommunications Division



Theodora Tzina
10715

Communication Systems II

Seeing Signals: Constellation Diagram Interpretation with CNN and
Vision-Language Models

Contents

1	Task 1: Literature Review	4
1.1	Automatic Modulation Classification with Deep Learning	4
1.2	The Role of Channel Impairments	4
1.3	Vision–Language Models for Domain-Specific Tasks	5
1.4	Parameter-Efficient Fine-Tuning	5
2	Task 2: Signal Model and Channel Impairments	6
2.1	Complex Baseband Representation and Energy Normalisation	6
2.2	Modulation Schemes	6
2.2.1	Amplitude-Shift Keying and Phase-Shift Keying	6
2.2.2	Rectangular and Cross QAM	7
2.2.3	Hexagonal QAM	7
2.2.4	Amplitude-Phase Shift Keying	7
2.3	Channel Impairment Models	8
2.3.1	Additive White Gaussian Noise	8
2.3.2	Phase Noise	8
2.3.3	IQ Imbalance	8
2.3.4	Continuous-Wave Jamming	9
2.3.5	Optional Amplitude Distortion	9
2.4	Impairment Cascade and Application Order	9
2.5	Discrete Severity Levels and Parameter Justification	9
2.6	Theoretical Symbol Error Probability of Square QAM	10
2.7	Monte Carlo SEP Simulation under Phase Noise	10
2.8	Summary of Implementation Artefacts	11
3	Task 3: Dataset Construction	12
3.1	Construction Pipeline	12
3.2	Clean Constellation Alphabets	12
3.3	Visual Signatures of Impairments	13
3.4	Image Rendering: From IQ Samples to Histogram Images	15
3.5	Label Structure and Question Vocabulary	16
3.6	Dataset Statistics and Splits	17
3.7	Output Artefacts and Reproducibility	18
4	Task 4: Model Architectures and Training	19
4.1	Shared Inputs and Outputs	19
4.2	CNN Baseline: Multi-Head ResNet18	19
4.2.1	Architecture	19
4.2.2	Multi-Task Loss	19
4.2.3	Training Procedure	20
4.3	Vision–Language Model: SmolVLM-256M with LoRA Adaptation	20
4.3.1	Model Selection Rationale	20
4.3.2	Architecture Overview	21
4.3.3	LoRA: Parameter-Efficient Fine-Tuning	21
4.3.4	Task Reformulation as Visual Question Answering	21
4.3.5	Training Procedure	22
4.4	Computational Limitations and Design Decisions	22
4.5	Summary of Implementation Files	24

5	Task 5: Experimental Results and Comparative Analysis	25
5.1	In-Distribution Classification Accuracy	25
5.2	Confusion Analysis	27
5.2.1	Modulation Confusion — CNN Baseline	27
5.2.2	Impairment-Task and SNR Confusion — CNN Baseline	28
5.2.3	Vision–Language Model Generation Behaviour	28
5.3	Robustness to Low SNR	30
5.4	Sensitivity to Impairment Severity	31
5.5	Generalisation Capability	32
5.5.1	Held-out SNR Bin	32
5.5.2	Intermediate-Impairment OOD	34
5.6	Computational Complexity	35
5.7	Summary and Conclusions	36

1 Task 1: Literature Review

This work sits at the intersection of three active research lines: machine learning for communications-signal analysis, multimodal foundation models for scientific domains, and parameter-efficient fine-tuning of pretrained transformers. We review each in turn, with particular attention to two pieces of prior work that directly motivate the project—the CNN-based modulation classifier of Oikonomou et al. [1], which defines the modulation set and impairment model adopted throughout this report, and RF-GPT [2], which proposes that vision-language models can be repurposed to interpret radio-signal representations through natural-language interfaces.

1.1 Automatic Modulation Classification with Deep Learning

Automatic modulation classification (AMC)—the task of identifying the modulation scheme of a received signal without prior knowledge of the transmitter’s configuration—has been studied for decades under both feature-engineered and learning-based paradigms. Classical approaches relied on higher-order statistics, cyclostationary feature detectors, and maximum-likelihood estimators that require accurate channel knowledge; deep-learning approaches have largely supplanted these by learning the relevant signal statistics directly from data. The seminal work of O’Shea et al. [3] demonstrated that a convolutional network operating on raw IQ time-series samples could outperform handcrafted features across a wide range of signal-to-noise ratios, opening a productive line of research on neural AMC.

Subsequent work has gradually moved from time-series CNNs to image-based architectures by rendering the IQ samples as two-dimensional constellation diagrams or density images. This re-framing allows established image-classification backbones, such as the residual networks of He et al. [4], to be transferred to the radio domain with minimal modification, and it places the classification problem in a representation that is interpretable to a human reader as well as to a model. The most directly relevant recent work in this image-based line is Oikonomou et al. [1], which uses a CNN to perform AMC under phase imperfections on the sixteen-modulation alphabet—spanning ASK, PSK, QAM, hexagonal QAM, and APSK variants—that we adopt unchanged in this project. Their results frame the difficulty of the task in a way that is directly comparable to ours and establish the impairment regime to which our own models must generalise.

1.2 The Role of Channel Impairments

A second strand of the literature concerns the impact of transceiver imperfections on the classifier’s performance. In an ideal channel limited only by additive Gaussian noise, modulation classification reduces to a problem of resolving symbol-energy patterns at a given signal-to-noise ratio, and high-order quadrature amplitude modulation schemes become increasingly difficult to identify as the noise variance approaches the squared inter-symbol distance. Real receivers, however, face additional impairments: phase noise from imperfect local oscillators, gain and phase mismatches between the in-phase and quadrature branches, occasional narrow-band interference from other transmitters, and saturation effects in the power amplifier chain. Each of these distorts the constellation in a characteristic geometric way, as illustrated in Section 3.3.

Of particular importance for our experimental setup is the wrapped-Gaussian phase-noise model, which introduces an *irreducible* symbol-error floor that does not vanish even as the SNR grows without bound. This phenomenon is central to the analysis of Oikonomou et al. [1] and is reproduced numerically by the Monte Carlo simulations of Section 2.7, where it is shown that higher-order QAM is disproportionately affected. The recognition that classifiers must remain robust under such impairments motivates the structured dataset construction of Task 3, in which each parameter axis (modulation, SNR, phase noise, IQ imbalance, jamming) is sampled independently.

1.3 Vision–Language Models for Domain-Specific Tasks

Outside the radio-signal domain, the past three years have seen the rapid maturation of vision–language models (VLMs)—large transformer architectures that ingest images and text jointly and produce textual outputs. These models, exemplified by the Idefics3 family of Laurencon et al. [5], are pretrained on massive corpora of image–caption and image–instruction pairs and inherit the zero- and few-shot generalisation properties of modern foundation models. While the flagship variants run to several billion parameters and are impractical to train or fine-tune on consumer hardware, recent work has produced significantly smaller distilled versions of the same architecture. SmolVLM-256M-Instruct [6], the model adopted in this project, retains the Idefics3 design while reducing the parameter count to a level compatible with single-GPU fine-tuning on free-tier cloud accelerators—a constraint we revisit in detail in Section 4.4.

The most directly relevant application of vision–language models to the radio domain is RF-GPT [2], which proposes that a multimodal model can be adapted to answer structured natural-language questions about visual representations of wireless signals, ranging from constellation diagrams to spectrograms. RF-GPT establishes the conceptual framework that the present work specialises: an image-based view of a received signal, a closed but linguistically natural set of questions, and a single multimodal model that produces structured answers from both inputs jointly. We follow this framing while restricting our scope to constellation imagery as the input modality and to the five-question vocabulary defined in Section 3.5 as the output target. This more restricted setup makes the comparison against a purely vision-based CNN baseline meaningful and reproducible within the available compute budget.

1.4 Parameter-Efficient Fine-Tuning

Adapting a pretrained vision–language model to a new domain—in our case, the recognition of communications-signal properties from constellation images—is in principle a standard transfer-learning task. In practice, however, the size of modern multimodal models makes full fine-tuning of all parameters prohibitively expensive: the optimiser state of an Adam-style optimiser alone consumes several times the memory of the model itself, and on the modest GPU resources available for this project it is simply infeasible. The dominant response to this constraint, in the broader transformer literature, is the family of parameter-efficient fine-tuning (PEFT) methods, of which low-rank adaptation [7] is the most widely deployed.

LoRA freezes the pretrained weights of selected linear layers and adds a pair of small trainable matrices whose product produces a low-rank update to the original weight matrix. Only these auxiliary matrices and their optimiser state are stored during training, reducing the trainable parameter count by two to three orders of magnitude with little impact on downstream accuracy in most settings. We use LoRA in the configuration described in Section 4.3 to fine-tune SmolVLM on the constellation dataset, updating roughly half a percent of the model’s parameters while keeping the remainder frozen at their pretrained values. The interaction between the small LoRA capacity, the limited single-epoch training budget that follows from the same compute constraints, and the downstream accuracy on a niche, out-of-distribution input domain is one of the central empirical questions that the comparative analysis of Task 5 aims to address.

Together, these four threads—image-based AMC, the role of transceiver impairments, vision–language models, and parameter-efficient fine-tuning—frame the experimental contribution of the present work. The remainder of the report develops a self-contained controlled dataset (Task 3) and exposes a fixed image classifier (Section 4.2) and a fine-tuned vision–language model (Section 4.3) to the same set of conditions, with the aim of quantifying how each architecture trades off accuracy, robustness, and computational cost on a task that lies at the intersection of communications signal processing and modern computer vision.

2 Task 2: Signal Model and Channel Impairments

This section establishes the mathematical framework on which the entire dataset construction rests. We begin with the complex baseband representation of digitally modulated signals and define the alphabets of the sixteen modulation schemes adopted from [1]. The four channel impairments introduced in the IQ domain—additive Gaussian noise, phase noise, IQ imbalance, and continuous-wave jamming—are then derived, each accompanied by the analytical model used in the implementation and a discussion of the assumptions made. We close with the closed-form symbol error probability of square QAM under additive noise and present the Monte Carlo simulation that quantifies the irreducible error floor caused by wrapped-Gaussian phase noise.

All signal-domain functionality described in this section is consolidated in a single file, `signals.py`. It exposes the modulation alphabets through `get_constellation(scheme)`, the impairment routines through `add_awgn`, `add_phase_noise`, `apply_iq_imbalance`, `add_cw_jammer`, and `apply_amplitude_distortion`, and a unified channel wrapper `apply_channel` that applies the impairments in the canonical order discussed in Section 2.4. Numerical parameter values for the severity levels and SNR bins are stored in `config.yaml` so that they can be modified without touching source code.

2.1 Complex Baseband Representation and Energy Normalisation

A digitally modulated bandpass signal centred at carrier frequency f_c is conveniently expressed in its complex baseband form,

$$x(t) = \Re \left\{ s(t) e^{j2\pi f_c t} \right\}, \quad (1)$$

where $s(t) = s_I(t) + j s_Q(t)$ collects the in-phase (s_I) and quadrature (s_Q) components into a single complex envelope. Sampling at the symbol rate, the transmitted stream reduces to a sequence of complex symbols $s_k \in \mathcal{S}$, where $\mathcal{S} \subset \mathbb{C}$ is a finite *constellation* of cardinality M . All signal generation in this project is performed directly in complex baseband form; no explicit carrier modulation is simulated. This methodological choice isolates the impairment effects from any carrier-related artefacts and aligns with the framework of [1].

Each alphabet is scaled so that the average symbol energy equals unity,

$$\mathbb{E} [|s|^2] = \frac{1}{M} \sum_{k=1}^M |s_k|^2 = 1. \quad (2)$$

This normalisation is fundamental to the experimental design: it ensures that the signal-to-noise ratio carries the same physical meaning across all sixteen modulations. Without it, the same nominal SNR would correspond to different ratios of useful-signal energy to noise variance depending on the alphabet, making any cross-modulation comparison ill-defined. The internal helper `_normalize_unit_energy` performs this scaling at construction time; the property is verified at startup by the self-test routine that prints the mean and peak squared modulus of every alphabet.

2.2 Modulation Schemes

The set of sixteen modulations specified in [1] spans four families differing in the geometry of their constellation. Below we summarise the construction rule for each family and the specific parameters chosen for the schemes adopted in this work. A summary appears in Table 1.

2.2.1 Amplitude-Shift Keying and Phase-Shift Keying

The 4-ASK and 8-ASK alphabets place M equally spaced points on the real axis,

$$\mathcal{S}_{M\text{-ASK}} = \{(2k - M + 1) : k = 0, 1, \dots, M - 1\}, \quad (3)$$

with subsequent unit-energy normalisation. BPSK is the binary case $M = 2$ with alphabet $\{-1, +1\}$. QPSK places $M = 4$ points on the corners of a unit-energy square, $\{(\pm 1 \pm j)/\sqrt{2}\}$.

2.2.2 Rectangular and Cross QAM

Square QAM constellations with $M \in \{16, 64, 256\}$ are formed as Cartesian products of two $\sqrt{M}$ -level ASK constellations along the I and Q axes,

$$\mathcal{S}_{M\text{-QAM}} = \left\{ a + jb : a, b \in \{\pm 1, \pm 3, \dots, \pm(\sqrt{M} - 1)\} \right\}. \quad (4)$$

For $M = 32$ and $M = 128$, square geometries are unavailable and we adopt the standard cross constellations: 32-QAM is obtained from a 6×6 grid with the four single corner points removed, while 128-QAM removes four 2×2 corner blocks from a 12×12 grid. The cross shape minimises the average energy under a fixed minimum-distance constraint and is the textbook choice for non-square QAM orders.

2.2.3 Hexagonal QAM

Hexagonal QAM places its $M \in \{4, 16, 64\}$ points on the triangular lattice generated by the basis vectors $v_1 = (1, 0)$ and $v_2 = (1/2, \sqrt{3}/2)$, retaining the M points closest to the origin:

$$\mathcal{S}_{M\text{-HQAM}} = \arg \min_{|\mathcal{S}|=M} \sum_{s \in \mathcal{S}} |s|^2, \quad \mathcal{S} \subset \mathbb{Z}v_1 + \mathbb{Z}v_2. \quad (5)$$

The triangular lattice is the densest two-dimensional packing of points and therefore achieves a lower average energy than the rectangular grid for the same minimum distance, at the cost of a more complex symbol-to-bit mapping.

2.2.4 Amplitude-Phase Shift Keying

APSK constellations place their points on concentric rings, parameterised by a tuple of radii $(r_1, \dots, r_L)$, points-per-ring $(n_1, \dots, n_L)$, and optional phase offsets $(\phi_1, \dots, \phi_L)$:

$$\mathcal{S}_{\text{APSK}} = \bigcup_{\ell=1}^L \left\{ r_\ell e^{j(\phi_\ell + 2\pi k/n_\ell)} : k = 0, \dots, n_\ell - 1 \right\}. \quad (6)$$

We adopt the DVB-S2X configurations for code rate 2/3: 16-APSK with (4, 12) points and radii (1, 2.85); 32-APSK with (4, 12, 16) and (1, 2.84, 5.27); 64-APSK with (4, 12, 20, 28) and (1, 2.6, 4.3, 6.0); and 128-APSK with (8, 16, 20, 28, 28, 28) and (1, 2, 3, 4, 5, 6). Phase offsets between successive rings are chosen to half the angular spacing of the inner ring to maximise the minimum inter-ring distance.

Table 1: Modulation alphabets used. All alphabets are normalised so that $\mathbb{E}[|s|^2] = 1$.

Family	Schemes	M	Geometry
ASK	4-ASK, 8-ASK	4, 8	Real axis, equispaced
PSK	BPSK, QPSK	2, 4	Unit-energy phasors
Rect. QAM	16, 64, 256-QAM	16, 64, 256	Square grid
Cross QAM	32, 128-QAM	32, 128	Square grid with corners trimmed
HQAM	4, 16, 64-HQAM	4, 16, 64	Hexagonal lattice
APSK	16, 32, 64, 128-APSK	16, 32, 64, 128	Concentric rings (DVB-S2X)

The dedicated builders `_rect_qam`, `_cross_qam`, `_hex_qam`, and `_apsk` implement the above constructions; the public entry point `get_constellation(scheme)` dispatches on the scheme string and returns the normalised complex alphabet.

2.3 Channel Impairment Models

We model four impairments acting on the transmitted symbol s , all operating directly in the IQ domain. Their analytical forms are given below; the order in which they are applied within the cascade is fixed in Section 2.4.

2.3.1 Additive White Gaussian Noise

The most universal channel impairment is the addition of complex Gaussian noise:

$$r = s + n, \quad n \sim \mathcal{CN}(0, \sigma^2), \quad (7)$$

with real and imaginary components each distributed as $\mathcal{N}(0, \sigma^2/2)$. Under the unit-energy convention of Eq. (2), the linear-units SNR equals the inverse of σ^2 , hence

$$\sigma^2 = 10^{-\text{SNR}_{\text{dB}}/10}. \quad (8)$$

The implementation in `add_awgn(s, snr_db, rng)` draws two independent standard-normal arrays for the real and imaginary parts and scales them by $\sigma/\sqrt{2}$.

2.3.2 Phase Noise

Phase noise originates from imperfect local oscillators in both the transmitter and the receiver. Following [1], we model it as a wrapped Gaussian random phase applied independently to each symbol,

$$y = s \cdot e^{j\theta}, \quad \theta \sim \mathcal{N}(0, \sigma_\varphi^2). \quad (9)$$

The wrapping is implicit, since $e^{j\theta}$ is 2π -periodic; for the standard deviations considered in this work ($\sigma_\varphi \leq 6^\circ$), the difference between a wrapped and an unwrapped Gaussian is numerically negligible.

We adopt the symbol-by-symbol (memoryless) model rather than a correlated random-walk Wiener process, again following [1]. This is appropriate when the phase-noise bandwidth is wide compared with the symbol rate, and is the model that admits the cleanest theoretical analysis (Section 2.7). A Wiener variant is also available in `add_phase_noise` through the `correlated=True` flag for future experiments, but is not used in the current dataset.

A central assumption of our analysis is that the receiver does *not* estimate or compensate for the carrier phase: the maximum-likelihood detector used for symbol error probability evaluation operates in the symbol space without phase tracking. This is consistent with the framing of [1] and makes the impact of phase noise directly visible in the SEP curves of Section 2.7.

2.3.3 IQ Imbalance

A real transmitter generates the bandpass signal by mixing s_I and s_Q with quadrature carriers,

$$x(t) = g_I s_I(t) \cos(2\pi f_c t + \phi_I) - g_Q s_Q(t) \sin(2\pi f_c t + \phi_Q). \quad (10)$$

In an ideal system, $g_I = g_Q = 1$ and $\phi_I = \phi_Q = 0$. In practice, mismatched gains ($g_I \neq g_Q$) and phase offsets ($\phi_I \neq \phi_Q$) between the I and Q branches produce *IQ imbalance*, which maps onto the analytical baseband relationship

$$y = \alpha s + \beta s^*, \quad (11)$$

with the conjugate term representing the image leakage characteristic of this impairment. The coefficients are

$$\alpha = \frac{1}{2} (g_I e^{-j\phi_I} + g_Q e^{-j\phi_Q}), \quad \beta = \frac{1}{2} (g_I e^{-j\phi_I} - g_Q e^{-j\phi_Q}). \quad (12)$$

We parameterise the imbalance by the amplitude mismatch ε and the phase mismatch ψ , split symmetrically between the two branches:

$$g_I = 1 + \frac{\varepsilon}{2}, \quad g_Q = 1 - \frac{\varepsilon}{2}, \quad \phi_I = -\frac{\psi}{2}, \quad \phi_Q = +\frac{\psi}{2}. \quad (13)$$

This symmetric splitting is a standard convention that recovers the ideal channel for $(\varepsilon, \psi) = (0, 0)$ and is symmetric in its two parameters. The function `apply_iq_imbalance(s, eps, psi_rad)` computes α and β from (ε, ψ) and applies Eq. (11) directly; when both parameters are zero, the input array is returned unmodified to avoid floating-point drift.

2.3.4 Continuous-Wave Jamming

To model coexistence with external interferers, we include a continuous-wave (CW) jammer at a normalised frequency offset f :

$$y_k = s_k + \sqrt{P_J} e^{j(2\pi f k + \phi_0)}, \quad (14)$$

where $\phi_0 \sim \mathcal{U}[0, 2\pi)$ is a random initial phase and P_J is the jammer power. The jammer-to-signal ratio is parameterised by the signal-to-interference ratio (SIR) in dB, with $P_J = 10^{-\text{SIR}_{\text{dB}}/10}$, again exploiting the unit-power signal convention.

The jammer is applied stochastically to a configurable fraction (25% by default) of the training samples, with SIR drawn uniformly from [5, 15] dB and $f \in [-0.4, 0.4]$ cycles per sample. The randomised inclusion is intentional: the dataset must teach the models to recognise whether interference is present rather than to treat its presence as a constant feature of the input.

2.3.5 Optional Amplitude Distortion

The Saleh AM/AM model is also implemented for amplifier-nonlinearity studies,

$$|y| = \frac{\alpha_a |s|}{1 + \beta_a |s|^2}, \quad (15)$$

with phase preserved (no AM/PM conversion). This impairment is disabled in the default dataset configuration; it is retained in the code for future experiments examining high-power amplifier saturation.

2.4 Impairment Cascade and Application Order

The four impairments are applied in the canonical order

$$\begin{aligned} s \rightarrow (\text{IQ imbalance}) \rightarrow (\text{AM/AM, optional}) \rightarrow (\text{phase noise}) \\ \rightarrow (\text{jammer}) \rightarrow (\text{AWGN}) \rightarrow r \end{aligned} \quad (16)$$

which models the physical signal path. IQ imbalance arises in the transmitter front-end and acts on the ideal symbol; amplitude distortion (when enabled) follows in the power amplifier; phase noise is contributed by both oscillators; external CW interference enters over the air; and thermal noise from the receiver front-end is added last. The wrapper function `apply_channel` enforces this sequence.

2.5 Discrete Severity Levels and Parameter Justification

Real-world impairments are continuous, but classification tasks require a finite label space. We discretise each impairment into three severity levels—*none*, *mild*, *severe*—chosen to span the physically realistic range while remaining distinguishable in the constellation image. The chosen values are summarised in Table 2.

Table 2: Discrete severity levels used in dataset construction. SNR is treated as a separate categorical task, partitioned into five bins.

Impairment	none	mild	severe
Phase noise std. deviation σ_φ (deg)	0	2	6
IQ amplitude mismatch ε	0	0.05	0.15
IQ phase mismatch ψ (deg)	0	2	8

SNR bin	Range (dB)
low	[0, 5)
low_mid	[5, 10)
mid	[10, 15)
high_mid	[15, 20)
high	[20, 30]

The phase noise standard deviations correspond to typical RF oscillator specifications: 2° rms phase jitter represents a moderately clean source, while 6° approaches the regime in which higher-order QAM becomes severely impaired (a point quantified in Section 2.7). The IQ imbalance values are likewise drawn from realistic transceiver tolerances: a 5% amplitude mismatch and 2° phase mismatch represent a well-calibrated direct-conversion front-end, whereas 15% and 8° represent a poorly compensated one. The SNR bins span the range from very noisy to almost noise-free conditions, with the `low_mid` bin reserved as held-out data for the out-of-distribution generalisation evaluation in Part II.

All numerical parameters are exposed through `config.yaml`, which the dataset-construction script reads at runtime. This makes the choice of severity levels a single-edit decision without modifying any source code, simplifying ablation studies.

2.6 Theoretical Symbol Error Probability of Square QAM

We derive the closed-form symbol error probability (SEP) of square M -QAM in AWGN as an analytical reference for the simulation results that follow. Treating the I and Q components as two independent $\sqrt{M}$ -PAM systems, each in additive Gaussian noise of variance $\sigma^2/2$, the per-component symbol error probability is

$$P_{\sqrt{M}} = 2 \left(1 - \frac{1}{\sqrt{M}} \right) Q \left(\sqrt{\frac{3 \text{SNR}}{M-1}} \right), \quad (17)$$

where $Q(x) = \frac{1}{2} \text{erfc}(x/\sqrt{2})$ is the Gaussian tail function. A symbol is correctly detected only if both components are correctly detected, hence

$$\text{SEP}_{M\text{-QAM}} = 1 - \left(1 - P_{\sqrt{M}} \right)^2. \quad (18)$$

This expression assumes maximum-likelihood detection with full knowledge of the alphabet and no phase noise. It serves as a baseline against which the wrapped-Gaussian simulations of the next subsection are compared.

2.7 Monte Carlo SEP Simulation under Phase Noise

While Eq. (18) gives the SEP under AWGN alone, no comparable closed-form expression exists for the case of wrapped-Gaussian phase noise. We therefore quantify the joint effect of σ_φ and SNR through Monte Carlo simulation. For each combination of modulation $M \in \{16, 64, 256\}$, phase-noise standard deviation $\sigma_\varphi \in \{0, 2, 4, 6\}^\circ$, and SNR value in $\{0, 2, 4, \dots, 30\}$ dB, the procedure draws 10^5 random symbols, passes them through the AWGN-plus-phase-noise channel,

and decides each received sample by nearest-neighbour search over the alphabet. The empirical error rate is recorded as the SEP for that operating point.

The decision rule is the maximum-likelihood detector in symbol space,

$$\hat{k} = \arg \min_{k \in \{1, \dots, M\}} |r - s_k|^2, \quad (19)$$

which is optimal under the assumption of no phase tracking at the receiver—precisely the scenario described in [1]. A phase-tracking receiver would, of course, lower the SEP curves; this comparison lies outside the scope of the present work.

The resulting curves are plotted in Figure 1. Three observations summarise their behaviour. First, at $\sigma_\varphi = 0^\circ$ the simulated points coincide with the analytical curve derived from Eq. (18), validating the implementation of both the channel and the detector. Second, increasing σ_φ produces an *irreducible* error floor that does not decrease with SNR. For 16-QAM the floor at $\sigma_\varphi = 6^\circ$ sits near 10^{-3} ; for 64-QAM near 10^{-1} ; for 256-QAM it rises above 0.3. This is the central observation of [1]: at fixed phase noise, higher-order QAM is disproportionately harmed because the minimum inter-symbol distance shrinks faster with M than the phase-noise tolerance. Third, the threshold at which phase noise dominates AWGN shifts to lower SNR for higher M : for 256-QAM with $\sigma_\varphi = 4^\circ$ the floor is reached around 20 dB, while for 16-QAM with the same σ_φ the curve still tracks the theoretical AWGN slope at the same SNR.

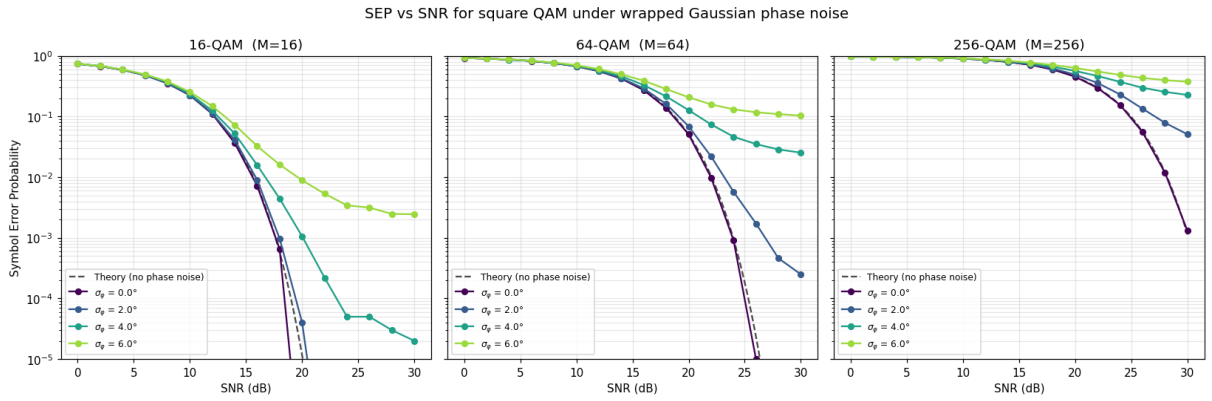


Figure 1: Symbol error probability versus SNR for square QAM under wrapped-Gaussian phase noise. The black dashed curve is the closed-form result of Eq. (18); coloured markers are the Monte Carlo estimates (100,000 symbols per point) for four values of the phase-noise standard deviation σ_φ . Higher-order QAM exhibits a more severe irreducible error floor and a smaller phase-noise tolerance.

The Monte Carlo procedure is implemented in `sep_vs_snr.py`. Running it as a standalone script produces both Figure 1 and the tabulated results in `results/sep_vs_snr.csv`.

2.8 Summary of Implementation Artefacts

The mathematical models of this section are realised in the following source files. `signals.py` contains the sixteen modulation alphabets (`get_constellation`), the random symbol generator (`generate_symbols`), and the impairment routines (`add_awgn`, `add_phase_noise`, `apply_iq_imbalance`, `add_cw_jammer`, `apply_amplitude_distortion`, and the unified `apply_channel` wrapper). The configuration file `config.yaml` stores the numerical values of the severity levels, SNR bins, jamming probability and frequency range, dataset size, and random seed. The standalone script `sep_vs_snr.py` carries out the Monte Carlo simulation that produces Figure 1. All scripts are deterministic for a fixed random seed and reproducible.

3 Task 3: Dataset Construction

Building on the signal model of Section 2.2 and the impairment cascade of Section 2.4, this section describes how individual IQ realisations are turned into a labelled dataset of constellation images suitable for both CNN and VLM training. We discuss the construction pipeline, the rendering procedure that maps IQ samples to fixed-size grayscale images, the labelling scheme, the partitioning strategy that supports both in-distribution and out-of-distribution evaluation, and the resulting dataset statistics. The construction is implemented in the standalone script `build_dataset.py`, which reads its parameters from `config.yaml` and produces three artefacts under `data/`: the image folder `images/`, the structured-label table `labels.csv`, and the question–answer corpus `qa_pairs.jsonl`.

3.1 Construction Pipeline

The script iterates over the Cartesian product of the four discretised parameter axes—modulation scheme, SNR bin, phase-noise severity, and IQ-imbalance severity—and draws a configurable number of independent realisations from each combination. For each draw, the procedure (i) samples a continuous SNR value uniformly within the active bin, (ii) maps the discrete severity labels to the corresponding numerical impairment parameters of Table 2, (iii) stochastically activates the CW jammer with configurable probability, (iv) generates the random symbol sequence and applies the impairment cascade defined in Section 2.4, (v) renders the resulting IQ samples into a fixed-size image, and (vi) writes one row to `labels.csv` together with five entries to `qa_pairs.jsonl`. The split assignment is performed inside the same loop and is discussed in Section 3.6.

The number of independent realisations per combination is set to five in the default configuration. With sixteen modulations, five SNR bins, three phase-noise levels, and three IQ-imbalance levels, this yields a total of $16 \times 5 \times 3 \times 3 \times 5 = 3600$ images and, consequently, $3600 \times 5 = 18\,000$ question–answer pairs.

Two design decisions deserve emphasis. First, the SNR value is drawn *continuously* within each bin rather than fixed at the bin midpoint; this prevents the model from associating each bin with a single SNR value and forces it to learn a range. Second, the random-number generator is seeded once at script entry, so the entire dataset is bit-for-bit reproducible from a single integer in `config.yaml`. Re-running the script with the same seed reproduces the same 3600 images, the same labels, and the same split assignment.

3.2 Clean Constellation Alphabets

Figure 2 displays the sixteen modulation alphabets generated by `get_constellation` for the schemes formalised in Section 2.2. The visualisation provides a sanity check that the mathematical constructions of that section produce the expected geometries: the ASK families collapse onto the real axis with equispaced points; BPSK and QPSK reduce to the simplest phase-keyed phasors; rectangular QAM forms regular square grids; cross QAM shows the trimmed-corner shape characteristic of 32 and 128 points; hexagonal QAM exhibits the triangular lattice with progressively more rings; and APSK consists of clearly separated concentric circles of points whose ring counts match the DVB-S2X specification.

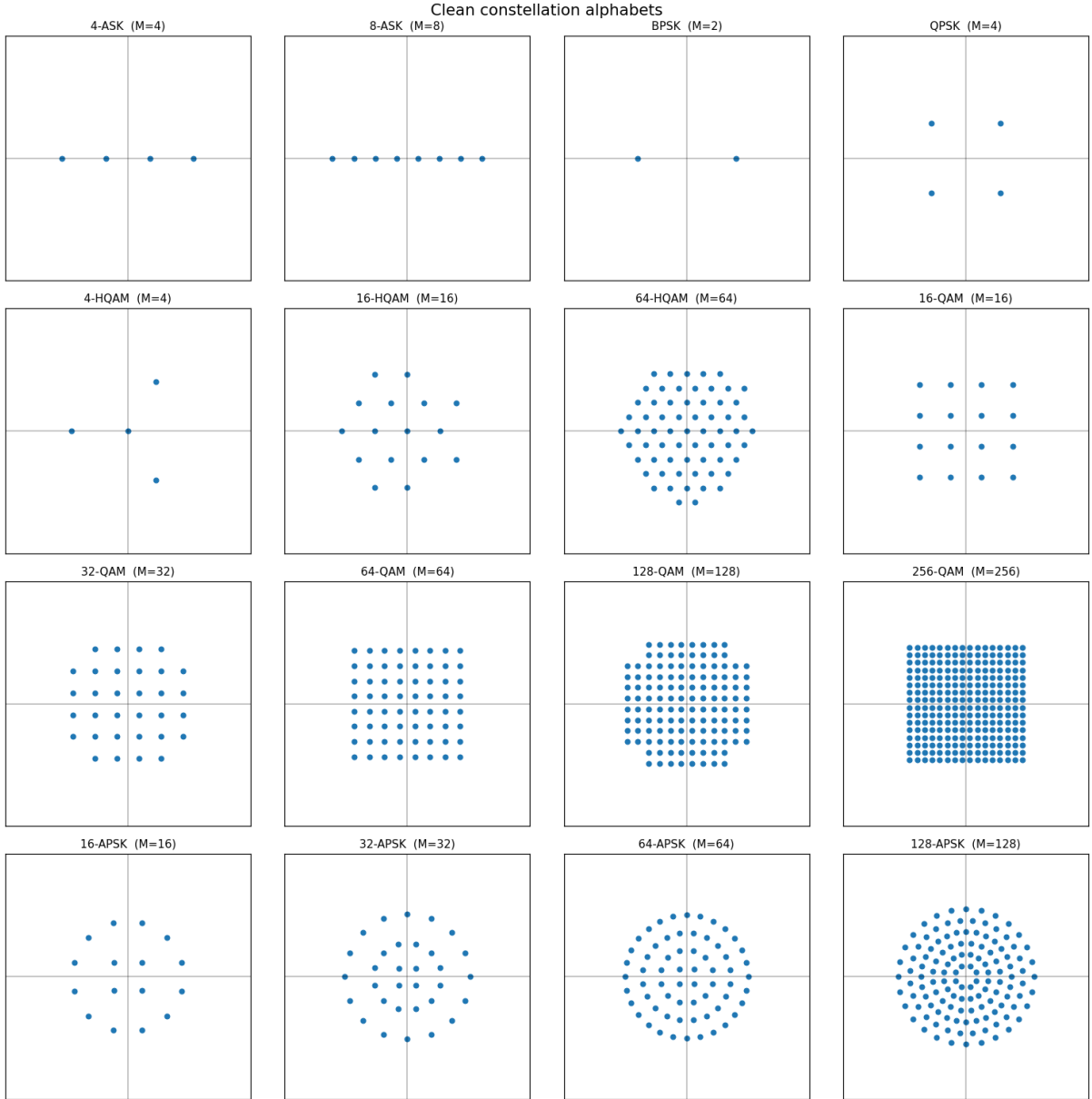


Figure 2: The sixteen normalised modulation alphabets used in this work. Each subplot shows the constellation points of one scheme with axes restricted to the $[-2.5, 2.5]$ range used throughout the dataset. All alphabets have $\mathbb{E}[|s|^2] = 1$, so their visual extent is directly comparable.

The figure also makes visible the intrinsic difficulty of the classification task. Several pairs of alphabets are geometrically similar at sufficient SNR—for example, 64-HQAM and 64-QAM occupy comparable regions of the I/Q plane with only the lattice angle differentiating them—yet must be distinguished by the trained models. Higher-order rectangular QAM (128, 256) becomes progressively denser, making robust classification dependent on resolving short inter-symbol distances; this is the regime that is most sensitive to the impairments studied in Section 2.7, and where the strongest interaction between modulation order and channel severity is expected.

3.3 Visual Signatures of Impairments

To verify that impairment models of Section 2.3 produce visually distinguishable distortions, we render representative IQ scatter plots for four reference modulations (QPSK, 16-QAM, 64-QAM, 16-APSK) under five impairment scenarios plus a clean baseline. The result is shown in Figure 3.

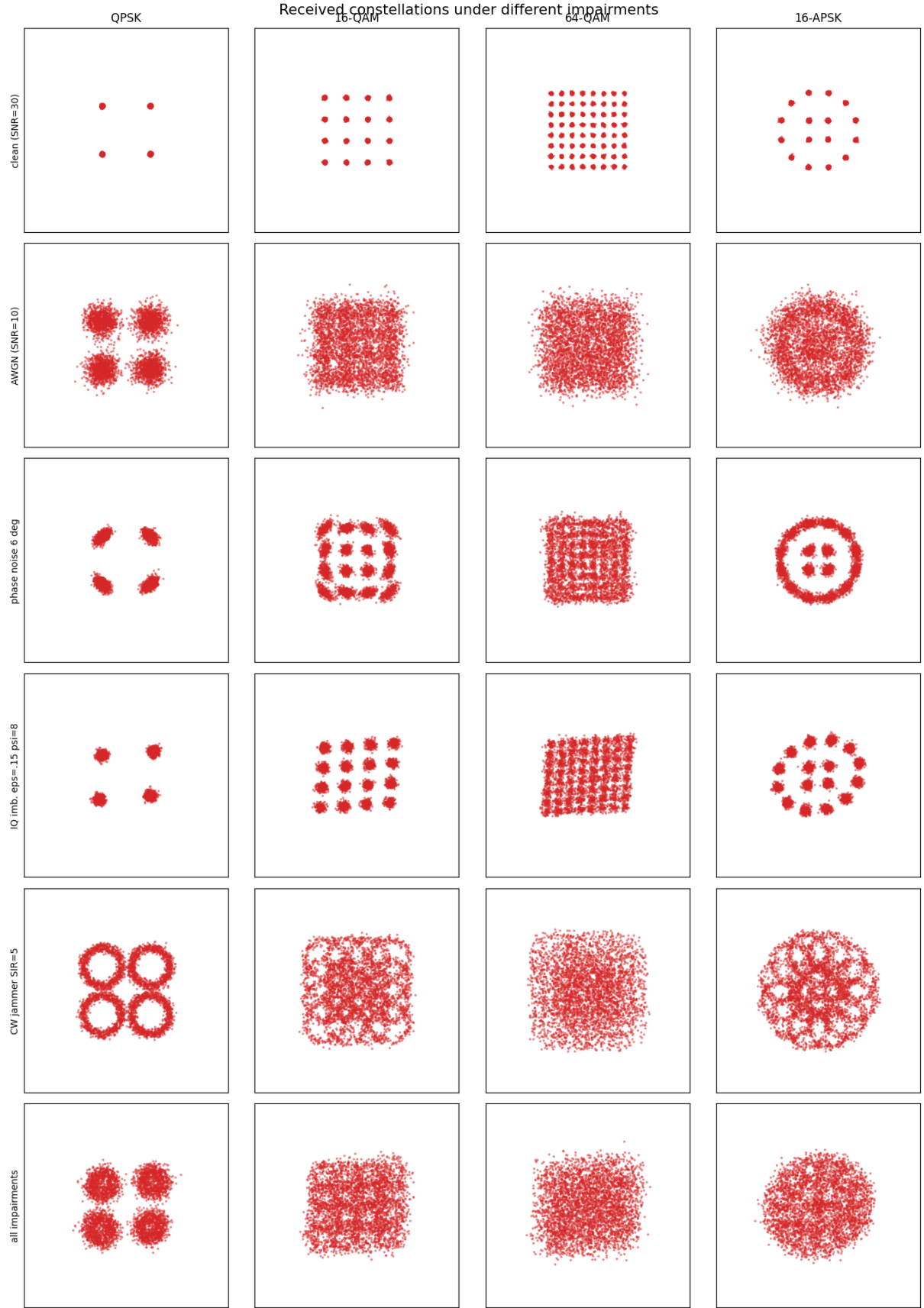


Figure 3: Received constellations under six channel scenarios, for four representative modulations. Each impairment leaves a qualitatively distinct fingerprint that the classification models must learn to associate with the corresponding label.

Each row of the figure isolates a single impairment, holding all others at zero, except for the final row that combines all impairments at moderate severity. Four observations are worth noting. First, AWGN produces an isotropic Gaussian cloud around each constellation point, with cluster radius governed by $1/\sqrt{\text{SNR}}$. Second, wrapped-Gaussian phase noise produces *arc-shaped* smearing along constant-radius contours: points at large $|s|$ smear more than points near the origin, since the linear arc length corresponding to a fixed phase deviation grows with the radius. The effect is particularly striking on 16-APSK, where the inner-ring points remain clustered while the outer-ring points smear into near-complete circles. Third, IQ imbalance produces a coherent affine deformation of the entire constellation: the QAM grids are sheared into parallelograms, with little random spread, since the impairment acts deterministically once (ε, ψ) are fixed. Fourth, the CW jammer produces a circular halo around each symbol, since its constant amplitude rotates uniformly in phase over the captured window. The bottom row aggregates all impairments at moderate severity and illustrates the type of input the trained models must process in the harshest dataset entries.

These qualitatively distinct fingerprints justify the choice of constellation imagery as the input modality: each label class corresponds to a different geometric pattern that is, in principle, recoverable from the image alone, without explicit access to the underlying IQ time series.

3.4 Image Rendering: From IQ Samples to Histogram Images

Each set of received IQ samples is rendered into a fixed-size grayscale image by the function `render_constellation` in `signals.py`. The rendering uses a two-dimensional density histogram rather than a scatter plot for three reasons. First, a density histogram is robust to the absolute number of symbols simulated: doubling the number of samples doubles the count per bin without changing the relative pattern, so the model’s input remains consistent across configurations of `n_symbols`. Second, the output is directly produced as a fixed-size array, avoiding the overhead and resolution variability of rendering a matplotlib figure. Third, the histogram naturally encodes the *density* of received symbols, which is the information-bearing quantity of the constellation, while a scatter plot at finite marker size discards density information above its saturation level.

The histogram procedure proceeds as follows. The IQ samples are clipped to the square $[-L, L]^2$ with $L = 2.5$, a region wide enough to contain almost every realisation given the unit-energy convention of Eq. (2). A regular grid of 224×224 bins is constructed over this square, and the joint histogram of the imaginary and real parts is computed. The image is flipped vertically so that positive quadrature appears at the top, matching the conventional constellation-diagram orientation. To compress the dynamic range, the histogram is passed through a $\log(1 + x)$ transformation: without this step, only the brightest pixels (the symbol centres at high SNR) would be visible, while the diffuse noise floor that carries low-SNR information would be lost in numerical zero. The transformed histogram is normalised to the $[0, 1]$ range, scaled to $[0, 255]$, and stored as an 8-bit grayscale PNG of size 224×224 .

The image size is chosen to match the standard input resolution of common pretrained image backbones, including the ResNet variants used by the CNN baseline of Task 4 [4] and the vision tower of the SmolVLM model used in Part II [6]. Adopting this resolution from the start eliminates the need for an additional resizing transform at training time, which would otherwise introduce a small loss of detail through bilinear interpolation. The view limit $L = 2.5$ is wide enough to encompass even severely impaired samples (e.g. heavy AWGN at $\text{SNR} = 0$ dB), yet tight enough that the symbol centres occupy a substantial fraction of the image, which improves the signal-to-background ratio of the input.

Figure 4 shows the resulting histogram images for the sixteen modulations under a moderate-impairment scenario ($\text{SNR} = 18$ dB, mild phase noise and IQ imbalance). The clear distinction between low-order modulations (ASK, BPSK, QPSK) and high-order ones (64, 128, 256-QAM, 64, 128-APSK) is visible, as is the gradual loss of internal structure as M increases. The images in Figure 4 are exactly what the CNN and VLM receive at training and inference time.

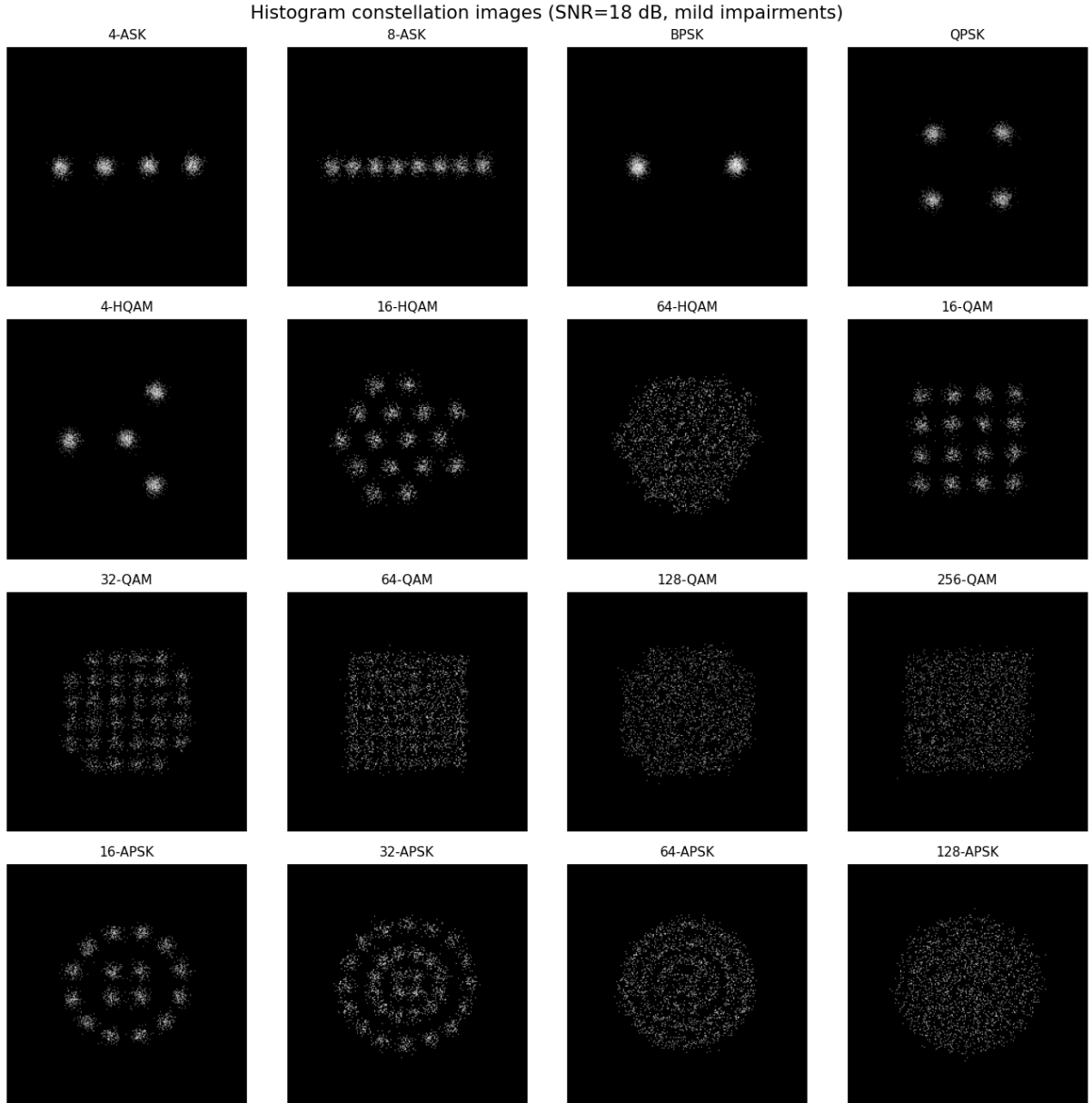


Figure 4: Histogram constellation images produced by the rendering pipeline, for the sixteen modulations under a mild-impairment, moderate-SNR scenario. Each panel is the 224×224 grayscale image fed directly to the CNN and to the vision encoder of the VLM.

3.5 Label Structure and Question Vocabulary

For every image, the construction script writes two complementary forms of supervision. The first, intended for the CNN baseline, is a row of structured labels in `labels.csv`: a flat CSV table with one row per image, listing both the discrete severity labels and the underlying numerical parameter values, together with split assignment and file paths. The second, intended for the VLM, is a set of natural-language question-answer pairs in `qa_pairs.jsonl`, with one JSON-lines record per (image, question) pair. The five questions and their answer vocabularies are listed in Table 3.

Table 3: Question–answer template used by the two label representations. The CNN treats each task as a discrete classification problem and reads the structured columns of `labels.csv`; the VLM is trained to generate the answer text directly given the image and the question, reading the question–answer pairs from `qa_pairs.jsonl`.

Question	Answer vocabulary
<i>What modulation is used?</i>	one of the 16 schemes (4-ASK, 8-ASK, BPSK, QPSK, 4-HQAM, . . . , 128-APSK)
<i>What is the level of phase noise?</i>	none, mild, severe
<i>What is the level of I/Q imbalance?</i>	none, mild, severe
<i>Is there external interference?</i>	yes, no
<i>What is the SNR range?</i>	low, low_mid, mid, high_mid, high

Two aspects of this design merit explanation. First, every task uses a *disjoint* answer vocabulary even when two tasks share a conceptually similar scale. Phase noise and IQ imbalance are both graded as `none/mild/severe`, but the question text disambiguates the target task. This is essential for the VLM, which generates the answer as free text: were the same words used in two different question contexts, the model would face an unresolvable ambiguity at generation time. The CNN, which reads the structured columns directly, is not affected by this consideration; the consistent vocabulary is therefore a constraint imposed by the VLM that we propagate to the CNN labels for symmetry between the two pipelines.

Second, the choice of natural-language phrasing for the VLM questions is deliberate. The questions are written as full English interrogatives rather than as terse tokens (e.g. `[MOD]` for modulation, `[PN]` for phase noise) to keep the VLM operating in the linguistic regime in which its base model was pretrained [5, 6], which we expect to improve sample efficiency during fine-tuning.

A representative excerpt from `qa_pairs.jsonl` for a single image is shown below.

```
{
  "image": "data/images/000123.png",
  "question": "What modulation is used?",
  "answer": "QPSK"
}
{
  "image": "data/images/000123.png",
  "question": "What is the level of phase noise?",
  "answer": "mild"
}
{
  "image": "data/images/000123.png",
  "question": "What is the level of I/Q imbalance?",
  "answer": "none"
}
{
  "image": "data/images/000123.png",
  "question": "Is there external interference?",
  "answer": "no"
}
{
  "image": "data/images/000123.png",
  "question": "What is the SNR range?",
  "answer": "high_mid"
}
```

The companion `labels.csv` row for the same image stores the discrete labels (`modulation`, `phase_noise_level`, `iq_imbalance_level`, `jamming`, `snr_bin`) together with the underlying numerical values (`snr_db`, `phase_noise_std_deg`, `iq_eps`, `iq_psi_deg`, `jam_sir_db`), the file path, and the split assignment. The numerical columns are not used by the CNN training loop (which reads only the categorical labels), but they are kept for two reasons: they enable post-hoc analyses such as the accuracy-versus-SNR curves of Task 5, and they allow the same CSV to be re-used by future experiments that may operate on continuous targets.

3.6 Dataset Statistics and Splits

The full dataset consists of 3600 images distributed evenly across the sixteen modulations: each modulation is represented by exactly 225 images, by construction of the parameter grid. The dataset is partitioned into four splits—`train`, `val`, `test`, and `ood_test`—with the assignment rule designed to support two distinct notions of generalisation evaluation.

The first three splits sample from the same impairment-parameter distribution and serve the standard roles of in-distribution training, hyperparameter selection, and held-out evaluation. The fourth split, `ood_test`, is reserved for evaluating *out-of-distribution generalisation*: it consists of all images whose SNR was drawn from the `low_mid` bin, which the training procedure never observes. This bin (5–10 dB) was chosen rather than one of the SNR extremes because it lies between two training bins (`low`: 0–5 dB and `mid`: 10–15 dB), so a model that has truly learned a continuous SNR mapping should be able to interpolate, while one that has merely memorised the training bins will fail to generalise.

Concretely, the assignment proceeds as follows. Any image whose SNR bin equals `low_mid` is routed directly to `ood_test`, regardless of any other parameter. The remaining 2880 images are distributed among `train`, `val`, and `test` by drawing an independent uniform random number per image and comparing against thresholds set by the configured ratios (70/15/15). Resulting split sizes for the default configuration are summarised in Table 4.

Table 4: Dataset split sizes for the default configuration. The `low_mid` SNR bin is held out entirely for the out-of-distribution evaluation in Task 5; the remaining bins are partitioned in a 70/15/15 ratio between `train`, `val`, and `test`.

Split	Images	Composition
<code>train</code>	2007	$\approx$ 70% of the four in-distribution SNR bins
<code>val</code>	423	$\approx$ 15% of the same
<code>test</code>	450	$\approx$ 15% of the same
<code>ood_test</code>	720	all images with SNR bin <code>low_mid</code> (5–10 dB)
Total	3600	

Within the training split, the empirical SNR distribution is centred at 14.4 ± 8.5 dB (mean $\pm$ standard deviation), reflecting the union of the four retained bins (0–5, 10–15, 15–20, and 20–30 dB). Each of these four bins contributes a comparable number of samples to training, and each of the sixteen modulations is represented in every split, ensuring that the held-out splits contain examples of every class. A second, separate generalisation evaluation—at *intermediate impairment levels* drawn from values not present in the training severity table—is performed at evaluation time using a freshly generated companion dataset, and is discussed together with its results in Task 5.

3.7 Output Artefacts and Reproducibility

The dataset construction produces three artefacts under `data/`. The folder `data/images/` contains 3600 PNG files, each a 224×224 grayscale histogram image, named sequentially as `000000.png` through `003599.png`. The file `data/labels.csv` is a comma-separated table with one row per image and fourteen columns covering identifiers (`image_id`, `filename`, `image_path`), the categorical labels described in Section 3.5, the underlying numerical parameter values, and the split assignment. The file `data/qa_pairs.jsonl` is a JSON-lines file with 18 000 records (five per image), each containing the image path, the question text, and the answer string.

All file paths recorded in `labels.csv` and `qa_pairs.jsonl` are stored as project-relative paths with forward slashes (e.g. `data/images/000123.png`), which makes the dataset portable across operating systems and execution environments. The image paths are resolved at runtime by a helper in `signals.py` that joins them with the project’s script directory, so the same dataset files can be moved between a local Windows workstation and a Linux cloud-GPU environment without modification.

The entire dataset is produced by a single command, `python build_dataset.py`, and is reproduced bit-for-bit by re-running the same command with the same seed. The total runtime is approximately 15–30 seconds on a modern laptop CPU.

4 Task 4: Model Architectures and Training

This section presents the two architectures evaluated on the dataset constructed in Task 3. The CNN baseline, described in Section 4.2, is a multi-head convolutional network trained from a pretrained image backbone; it serves as a purely vision-based reference for the comparison. The vision-language model (VLM), described in Section 4.3, is a small open-source multimodal model adapted to the task with parameter-efficient fine-tuning. Both models are trained on the same images and held-out splits, but they differ fundamentally in how they ingest the task: the CNN predicts a set of categorical logits from the image alone, while the VLM receives the image alongside a natural-language question and produces the answer as generated text. Section 4.4 discusses the computational constraints that influenced several design decisions, as required by the project specification.

4.1 Shared Inputs and Outputs

Both models read the constellation images produced by the pipeline of Section 3.4: 224×224 grayscale histograms stored as 8-bit PNGs and replicated to three channels at load time so that they can be consumed by image backbones pretrained on natural RGB photographs. Both models train on the same `train` split and select checkpoints using the same `val` split; both are evaluated on the same `test` and `ood_test` splits described in Section 3.6. The set of supervised targets is also shared, with one minor extension: the VLM additionally learns to answer the jamming question (`yes/no`), which the CNN does not predict by default. The comparison in Task 5 therefore focuses on the four tasks common to both models (modulation, phase noise, IQ imbalance, SNR bin), with the VLM’s jamming-detection accuracy reported separately.

4.2 CNN Baseline: Multi-Head ResNet18

4.2.1 Architecture

The CNN baseline reuses a ResNet18 backbone [4], the smallest member of the ResNet family, as a generic image-feature extractor. The choice of ResNet18 over a more recent or larger backbone is deliberate: it offers a well-understood baseline that converges quickly on small datasets, fits comfortably on a single T4 GPU together with batch sizes large enough for stable training, and is supported by mature pretrained weights from ImageNet that bootstrap the feature extractor at no extra cost. The original final classification layer is removed, leaving a 512-dimensional feature vector at the network’s penultimate layer.

Four task-specific heads are attached to this feature vector. Each head is a single linear layer mapping the 512-dimensional shared representation to the logits of one task: 16 classes for modulation, 3 for phase-noise level, 3 for IQ-imbalance level, and 4 for SNR bin (the `low_mid` bin is excluded from the SNR head’s output space, since it is held out for OOD evaluation). The total trainable parameter count is 11.2×10^6 , virtually all of which sits in the ResNet18 backbone; the four heads together account for fewer than 20 thousand parameters. All weights—including the backbone—are updated during training; we did not freeze any layer, since the source domain of the backbone (natural photographs) differs sufficiently from constellation histograms that full fine-tuning is justified.

4.2.2 Multi-Task Loss

Each task contributes an independent cross-entropy term to the loss. Letting $\hat{y}_t$ and y_t denote the logits and ground-truth label for task t , the per-sample loss is

$$\mathcal{L} = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} m_t \cdot \text{CE}(\hat{y}_t, y_t), \quad (20)$$

where $\mathcal{T} = \{\text{modulation, phase, iq, snr}\}$ and m_t is a binary mask that disables the SNR term for samples whose SNR bin is absent from the training distribution (i.e. the OOD validation samples that may appear in mixed loaders). The four tasks are weighted equally; a more elaborate weighting scheme was not necessary, since the per-task losses remained of comparable magnitude throughout training. The averaging over $|\mathcal{T}|$ keeps the overall scale of the gradient independent of the number of tasks, which simplifies the choice of learning rate.

4.2.3 Training Procedure

The model is trained for 15 epochs with the AdamW optimiser at an initial learning rate of 10^{-3} and weight decay 10^{-4} , with a cosine annealing schedule that brings the learning rate to zero by the final epoch. Mini-batches of 32 images are drawn at random from the training split; the validation split is evaluated after every epoch. The checkpoint with the highest average validation accuracy (across the four tasks) is retained for testing.

The training procedure is deterministic given the random seed in `config.yaml`. On a single NVIDIA T4 GPU (the configuration used for the experiments of Task 5), each epoch completes in approximately 7.9 seconds; the full 15-epoch run finishes in 118 seconds. The inference latency, measured by running the trained model over the test set, is 1.73 milliseconds per image, dominated by the forward pass through the ResNet18 backbone. Both numbers are reported in detail in Section 4.4 and compared to the corresponding VLM figures.

The training loop, the loss function, the validation logic, and the figure-generation code are implemented in `train_cnn.py`, which produces `cnn_metrics.json` together with all the CNN-related figures referenced in Task 5.

4.3 Vision–Language Model: SmolVLM-256M with LoRA Adaptation

4.3.1 Model Selection Rationale

The project specification requires a *lightweight* open-source vision–language model from the Hugging Face Hub and provides SmolVLM-256M-Instruct as one acceptable example. After surveying several candidates—including Idefics3-8B, PaliGemma-3B, and LLaVA-1.5-7B—we adopted SmolVLM-256M-Instruct [6] as the model used throughout the experiments, for four reasons:

- *Parameter count.* At 258×10^6 parameters, SmolVLM is the only model in the survey that fits entirely on a single NVIDIA T4 GPU (16 GB VRAM, the largest free-tier accelerator available) in `bf16` with non-trivial batch sizes, leaving enough memory for gradient checkpointing and Adam-style optimiser state.
- *Multimodal pretraining.* The model is initialised from a vision–language pretraining regime aligned with the task we are asking it to learn: associating image content with textual answers in English [5].
- *Open availability and tooling support.* SmolVLM is freely available on the Hugging Face Hub and is supported by the `transformers` library, which provides robust implementations of its processor (image and text tokenisation), generation loop (greedy and sampling decoders), and weight loaders.
- *Suitability for parameter-efficient fine-tuning.* The model’s language tower exposes the standard attention projection matrices (`q_proj`, `k_proj`, `v_proj`, `o_proj`), which are the canonical targets for LoRA injection.

We acknowledge that SmolVLM-256M is at the lower end of the capability spectrum of current VLMs and that its absolute accuracy will not be competitive with frontier models. The objective of this work, however, is not to push absolute performance but to characterise the qualitative properties of the vision–language paradigm under realistic compute constraints. The chosen

model is faithful to the framing of the project specification, which positions the VLM as an experiment in architectural transfer rather than a production-grade classifier.

4.3.2 Architecture Overview

SmolVLM-256M is built on the Idefics3 architecture [5], which composes three principal modules. A *vision encoder*, a SigLIP-derived patch-based transformer, ingests the 224×224 RGB image and produces a sequence of patch embeddings. A *vision-language connector*, implemented as a small projection plus a pixel-shuffle operation, maps these embeddings into the language model’s embedding space and reduces their number to match a target token budget. The *language model*, a SmolLM-based decoder-only transformer, then ingests the visual tokens together with the textual prompt (a chat-formatted question) and produces an autoregressive sequence of output tokens. The full model has 257.8×10^6 parameters, of which roughly 90% live in the language tower.

For training, we freeze the vision encoder entirely. The image features it produces from constellation histograms are already informative, and adapting them would not only increase the trainable parameter count by an order of magnitude but also pose a substantial overfitting risk on a dataset of only a few thousand images. The vision-language connector and the language tower are accessible for adaptation, but we update only a small fraction of their parameters through the LoRA mechanism described next.

4.3.3 LoRA: Parameter-Efficient Fine-Tuning

Full fine-tuning of a 258-million-parameter model on a small dataset is both computationally expensive and prone to catastrophic forgetting of the model’s pretrained capabilities. We instead use low-rank adaptation [7], which inserts a pair of trainable low-rank matrices into each selected linear layer while keeping the original weights frozen. For a frozen weight matrix $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, LoRA replaces W with the effective matrix

$$W' = W + \frac{\alpha}{r} BA, \quad (21)$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$ are the trainable low-rank factors and $r \ll \min(d_{\text{in}}, d_{\text{out}})$ is the rank. The scaling factor α/r keeps the magnitude of the update independent of the chosen rank. At inference, the trained BA product can be folded back into W at zero additional cost, but during training only the $r(d_{\text{in}} + d_{\text{out}})$ parameters of A and B require gradient computation and optimiser state.

Our configuration uses rank $r = 8$, scaling $\alpha = 16$, and a dropout of 0.05 applied to the LoRA inputs as a regulariser. The adapter targets the four standard attention-projection layers within every transformer block of the language tower: the query, key, value, and output projections. The vision encoder is excluded, and so is the vision-language connector, which we found to be small enough that even full updates would not exceed the parameter budget meaningfully and which we leave frozen for simplicity. With this configuration, the total trainable parameter count is 1.36×10^6 , or 0.53% of the model’s total 257.8×10^6 parameters—a $190\times$ reduction in optimiser state compared with full fine-tuning.

4.3.4 Task Reformulation as Visual Question Answering

Unlike the CNN, which produces a fixed set of logits, the VLM operates by generating natural-language text. The four classification tasks of the project must therefore be reformulated as visual question-answering problems. We use the question-answer template defined in Section 3.5, illustrated in Table 3: each training example consists of one constellation image, one natural-language question, and the corresponding correct answer in the model’s vocabulary.

The training input is constructed by applying the model’s chat template, which interleaves system-prompt tokens, image-placeholder tokens, the question text, and a generation prompt. The expected answer text is appended and the cross-entropy loss is computed token-by-token; crucially, the loss is *masked* on every token that belongs to the system prompt, the image tokens, the question, or any padding, so that the model is rewarded only for producing the correct answer and never for memorising the prompt. This is the standard recipe for instruction tuning of multimodal models.

At inference time, the same chat template is built without the answer, the model performs greedy autoregressive decoding for up to 16 new tokens, and the generated text is parsed against the task-specific vocabulary by a lightweight post-processor that handles minor surface variations (case, whitespace, partial matches against the closed-vocabulary answer set). Examples falling outside the expected vocabulary are recorded as errors.

4.3.5 Training Procedure

The model is trained for a single epoch over the same ~ 2000 training images used by the CNN, with five questions per image yielding approximately 10 000 training examples per epoch. Training uses the AdamW optimiser with learning rate 5×10^{-5} (a value chosen to be conservative given the small trainable parameter count and the risk of destabilising the pretrained weights). Mini-batches contain two examples each, and gradient accumulation over eight steps yields an effective batch size of 16. Mixed-precision training in `bfloat16` is used throughout, and gradient checkpointing is enabled on the language tower to reduce the memory footprint of intermediate activations.

A summary of the training-time hyperparameters for both models is given in Table 5. The training loop is implemented in `train_vlm.py`; the standalone evaluation script `evaluate_vlm.py` reloads the saved adapter and produces the figures and metrics analysed in Task 5.

Table 5: Training-time hyperparameters and resulting parameter counts for the two models.

	CNN baseline	VLM (SmolVLM + LoRA)
Backbone	ResNet18 (ImageNet)	SmolVLM-256M-Instruct
Total parameters	11.2×10^6	257.8×10^6
Trainable parameters	11.2×10^6 (100%)	1.36×10^6 (0.53%)
Adaptation method	Full fine-tuning	LoRA, $r = 8$, $\alpha = 16$
Optimiser	AdamW	AdamW
Initial learning rate	10^{-3}	5×10^{-5}
LR schedule	cosine annealing	constant
Weight decay	10^{-4}	10^{-4}
Per-device batch size	32	2
Gradient accumulation	1	8
Effective batch size	32	16
Epochs	15	1
Mixed precision	none (fp32)	<code>bfloat16</code>
Gradient checkpointing	disabled	enabled

4.4 Computational Limitations and Design Decisions

The hyperparameter choices listed in Table 5 are not the result of an unconstrained optimisation: they reflect a set of compromises imposed by the compute resources available for this project. Because the project specification explicitly asks for a discussion of computational limitations, we describe them in some detail here so that the reader can correctly interpret the comparison reported in Task 5.

All training was carried out on free-tier cloud GPU services. The only accelerator consistently available to us was a single NVIDIA T4 with 15 GB of usable VRAM, accessible through Google Colab and Kaggle Notebooks. Neither service permits sustained long-running training: the Colab free tier disconnects sessions after approximately 90 minutes of perceived idleness and revokes GPU access after roughly 5 cumulative hours per day, while Kaggle Notebooks limit interactive sessions to 12 hours and aggregate GPU usage to about 30 hours per week. These limits made it impossible to schedule the VLM training as a single uninterrupted job without active monitoring, and an earlier Colab run was indeed lost mid-evaluation after a runtime restart wiped the session’s filesystem; the experiment was subsequently re-run on Kaggle, where output artefacts can be persisted through the *Save Version* mechanism.

Three concrete design decisions were forced by these constraints.

First, the VLM’s per-device batch size had to be reduced to 2. With the default batch size of 4, the model exceeds the 15 GB VRAM budget even with mixed precision enabled, because the vision encoder retains its activations for the loss backpropagation even though it is itself frozen—freezing a module does not free its activations during the backward pass. Enabling gradient checkpointing on the language tower, together with the smaller batch, brought the peak memory below the available budget at the cost of approximately a 30% increase in per-step time. Gradient accumulation over eight steps restored the effective batch size to a value comparable to the CNN baseline.

Second, the VLM was trained for a single epoch rather than the multiple epochs that would typically be advisable. At approximately 14.4×10^3 seconds of compute per epoch (Table 6), a two-epoch run would have approached the Colab daily quota; three or more epochs would not fit within a single Kaggle interactive session. The single-epoch budget was therefore chosen as a calibrated trade-off between adapter quality and the risk of losing all training work to a session disconnect. As a consequence, the training loss at the end of the VLM run remained elevated at approximately 4.87, suggesting that further epochs would likely improve the test accuracy. This is an important qualification of the Task 5 comparison: the CNN baseline is reported close to its converged state, while the VLM is reported at an early stage of its optimisation trajectory.

Third, the in-script evaluation step that follows the training loop had to be moved to a standalone script. The generation phase of the VLM evaluation uses the model in inference mode with longer effective sequences (image tokens plus prompt plus partial output), and although it does not require optimiser memory, the combined footprint of the model in inference mode plus a non-trivial evaluation batch exceeded what remained on the T4 after the training-mode model had been loaded. The script `evaluate_vlm.py` reloads the saved LoRA adapter into a fresh process with `eval_batch=1`, which keeps the peak memory well below the limit at the cost of a slower wall-clock evaluation. The trained adapter itself is small (~ 5 MB) and is preserved by the same *Save Version* mechanism that allowed the experiment to complete at all.

A side-by-side summary of the compute budgets of the two models is presented in Table 6.

Table 6: Compute resources used by the two models. Wall-clock times are measured on a single NVIDIA T4 GPU.

	CNN baseline	VLM (SmolVLM + LoRA)
Hardware	NVIDIA T4 (Kaggle)	NVIDIA T4 (Kaggle)
Total training time	118 s	14 423 s (≈ 4.0 h)
Time per epoch	7.9 s	14 423 s
Epochs completed	15	1
Inference latency per image	1.73 ms	3 168 ms
Peak VRAM	~ 3 GB	~ 14 GB

Two further observations are worth recording. First, the ratio of training times—approximately $120\times$ in favour of the CNN, per epoch—is a direct quantitative expression of the difference in

model size between the two architectures. This ratio is itself an experimental finding of interest for the comparative analysis of Task 5, since it represents the price paid in compute for the additional capabilities (multimodality, parameter-efficient adaptation, in-context interpretability) that the VLM brings to the problem. Second, the inference latency of the VLM exceeds the CNN’s by approximately a factor of 1830 (1.73 ms versus 3 168 ms per image), owing to the autoregressive generation loop that the CNN does not have. The reader should keep this asymmetry in mind when interpreting the accuracy comparisons of Task 5: equality in accuracy between the two models would not imply equality in deployment cost.

4.5 Summary of Implementation Files

The training and evaluation procedures of this section are realised in three source files. `train_cnn.py` implements the multi-head CNN baseline, including the model definition (`MultiHeadCNN`), the multi-task loss of Eq. (20), the training loop with cosine learning-rate scheduling, the best-checkpoint selection logic, and the generation of all CNN-related figures and metrics. `train_vlm.py` implements the VLM fine-tuning pipeline, including the chat-template construction, the LoRA injection with the configuration described in Section 4.3, the masked instruction-tuning loss, gradient checkpointing, and the post-training adapter save. `evaluate_vlm.py` is the standalone evaluation script that reloads the trained adapter and produces the test- and OOD-split metrics, the inference-latency measurement, the severity-sensitivity analysis, and all VLM-related figures used in Task 5. All three scripts share helper utilities from `signals.py` and read their parameters from `config.yaml`, maintaining the single-source-of-truth principle established in Task 3.

5 Task 5: Experimental Results and Comparative Analysis

This section presents the experimental comparison of the two models described in Task 4, applying the five evaluation criteria specified in the project: classification accuracy on the in-distribution test set, robustness to low signal-to-noise ratio, sensitivity to impairment severity, generalisation to data drawn from outside the training distribution, and computational complexity. The results reported below are produced by the standalone evaluation scripts `evaluate_vlm.py`, `ood_intermediate.py`, and `compare.py`, and all figures referenced here are saved in the project’s `results/` directory.

The headline finding of the experiments, which structures the discussion that follows, is that the multi-head CNN baseline successfully learns all four shared classification tasks at non-trivial accuracy, while the LoRA-fine-tuned VLM *fails to converge to the task distribution* within the single-epoch training budget imposed by the compute constraints discussed in Section 4.4. The two architectures therefore occupy qualitatively different operating points in the comparison: the CNN is reported close to its converged behaviour, whereas the VLM is reported at an early, pre-convergent stage that we characterise in detail below. We retain the side-by-side presentation throughout, since the asymmetry itself is the comparative outcome that the project specification asks us to study.

5.1 In-Distribution Classification Accuracy

The first comparison evaluates per-task accuracy on the in-distribution test set, consisting of 450 images drawn uniformly from the four training SNR bins (the `low_mid` bin is held out for the OOD evaluation of Section 5.5). The CNN is evaluated on the four shared tasks; the VLM is evaluated on the same four tasks plus the jamming-detection question, which the VLM additionally learns. Numerical results are summarised in Table 7 and visualised in Figure 5.

Table 7: In-distribution test-set accuracy for the two models on the four shared tasks (modulation, phase noise, IQ imbalance, SNR bin) and the VLM-only jamming-detection task. Test set size: 450 images for the CNN; the VLM is evaluated on a random subsample of 200 images per the constraints of Section 4.4.

Task	CNN baseline	VLM (SmolVLM + LoRA)
Modulation (16 classes)	0.780	0.000
Phase-noise level (3 classes)	0.582	0.000
IQ-imbalance level (3 classes)	0.831	0.000
SNR bin (4 classes)	0.924	0.000
Jamming presence (2 classes)	—	0.795
Mean (four shared tasks)	0.779	0.000

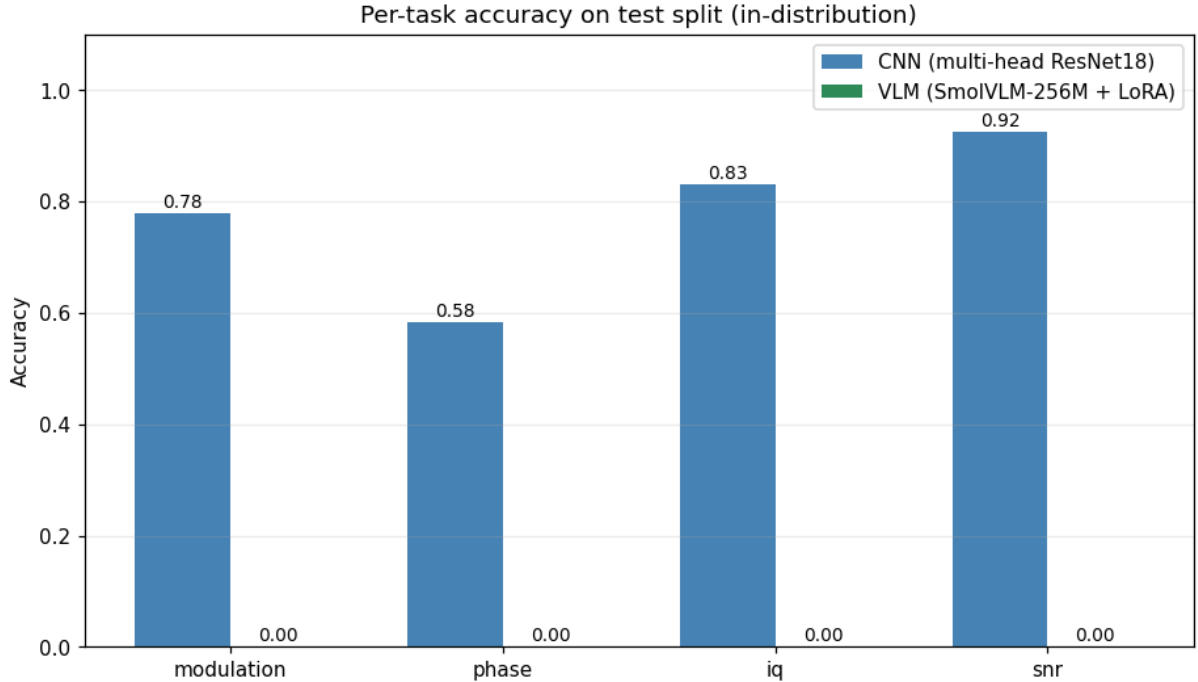


Figure 5: Per-task accuracy on the in-distribution test split. The CNN baseline (blue) achieves a mean accuracy of 77.9% across the four shared tasks; the VLM (green) achieves zero accuracy on all four, the result of incomplete convergence under the single-epoch training budget (Section 4.4).

The CNN baseline produces a regular pattern of results consistent with the difficulty hierarchy of the four tasks. The SNR-bin task is the easiest at 92.4%, since the underlying continuous SNR correlates strongly with the visible noise spread in the constellation image and the four-class label space is coarse. The IQ-imbalance task follows at 83.1%: as discussed in Section 3.3, IQ imbalance produces a coherent affine deformation that the network learns to read off the global geometry of the constellation. The modulation task, despite having 16 classes, is also classified at 78.0%, with most of the residual error concentrated on a few high-order families that we examine in Section 5.2. The phase-noise task is the most difficult, with an accuracy of 58.2%: the boundary between the *none* ($\sigma_\varphi = 0^\circ$) and *mild* ($\sigma_\varphi = 2^\circ$) classes is geometrically subtle and persistently confused even at moderate SNR, an effect we revisit through the confusion matrices in Section 5.2.

The VLM result requires more careful interpretation. The reported zero-accuracy figure on each of the four closed-vocabulary tasks is not a near-zero accuracy that has been rounded down: it reflects the fact that the model essentially never produces a string belonging to the corresponding answer vocabulary. The only above-zero accuracy in Table 7 is the jamming task at 79.5%, but this number is itself misleading. The dataset assigns jamming with probability 0.25, so a constant predictor that always answers “no” achieves an accuracy of approximately 0.75. The VLM’s 0.795 is statistically indistinguishable from this majority-class baseline, suggesting that the model has learned to default to the more frequent yes/no answer for the binary question while failing entirely on the four closed-vocabulary, multi-class tasks. The mechanism underlying this behaviour becomes immediately apparent when the model’s actual outputs are inspected, as we do in Section 5.2.

5.2 Confusion Analysis

5.2.1 Modulation Confusion — CNN Baseline

The modulation confusion matrix of the CNN baseline on the in-distribution test set is shown in Figure 6. Three structural observations emerge.

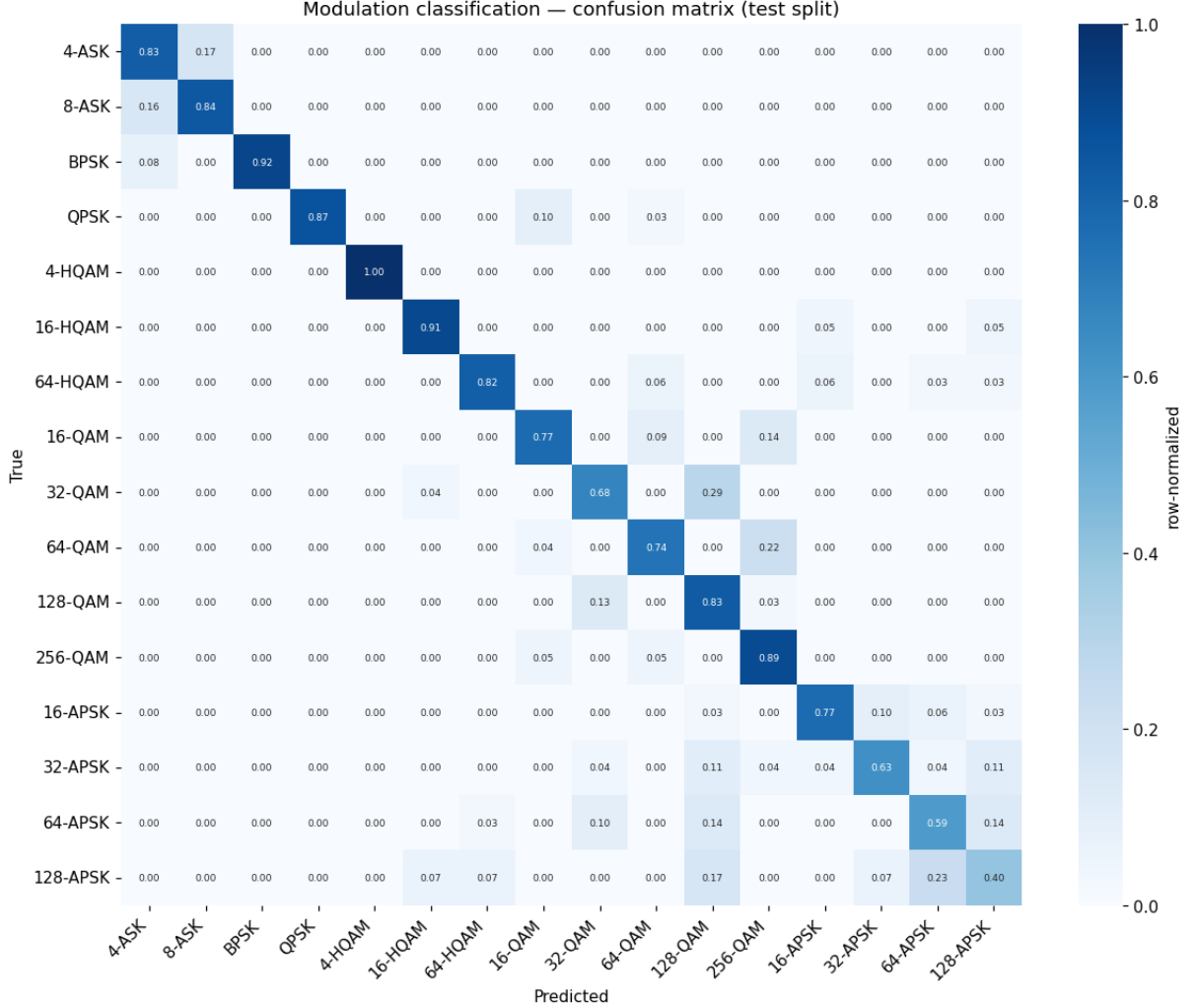


Figure 6: Row-normalised modulation confusion matrix for the CNN baseline on the in-distribution test split. The diagonal is dominant for low-order modulations and gradually degrades for higher-order ones, with confusions concentrated between adjacent orders within the same family.

First, low-order modulations are classified with very high accuracy: BPSK at 0.92, QPSK at 0.87, and 4-HQAM at 1.00, since their constellation images contain a small number of well-separated clusters that remain unambiguous even under non-trivial impairments. Second, the dominant residual confusion is between modulations of *similar order and geometry within the same family*: 32-QAM and 128-QAM (0.29 misclassification), 64-QAM and 128-QAM (0.22), 128-APSK and 64-APSK (0.23). These confusions are geometrically natural—adjacent rectangular-QAM orders share the same underlying lattice and differ mainly in their inter-symbol distance, which becomes harder to resolve as the constellation becomes denser. Third, cross-family confusions are rare (e.g. 16-QAM is essentially never mistaken for 16-APSK, despite both having $M = 16$), indicating that the rendering pipeline preserves the family-level geometric signature even when individual symbols overlap. The worst-classified scheme is 128-APSK at 0.40: its

outer rings of 20 and 28 points are densely populated and difficult to distinguish from those of 64-APSK at finite SNR.

5.2.2 Impairment-Task and SNR Confusion — CNN Baseline

The confusion matrices for the three remaining CNN tasks are shown in Figure 7.

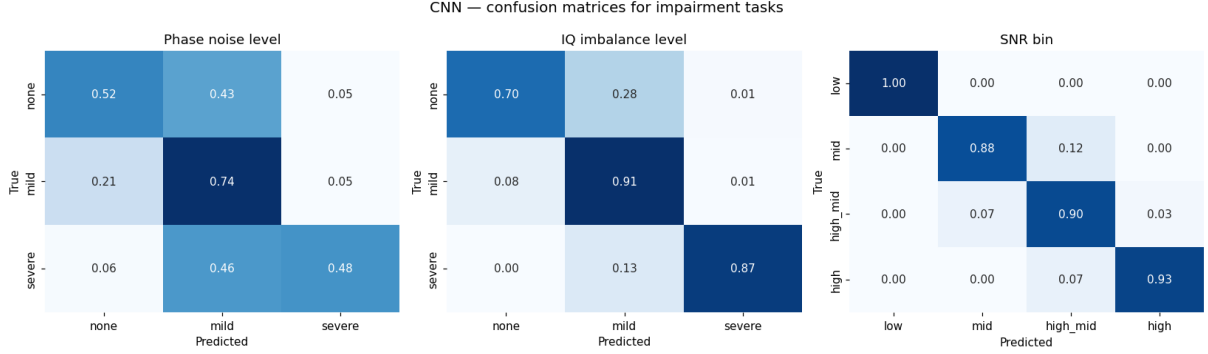


Figure 7: Row-normalised confusion matrices for the three remaining CNN tasks: phase-noise level (left), IQ-imbalance level (middle), and SNR bin (right). The SNR task is by far the easiest; phase-noise classification shows the largest residual confusion, concentrated between adjacent severity classes.

The SNR-bin task is classified with near-perfect accuracy across all four in-distribution bins (1.00, 0.88, 0.90, 0.93), with the small amount of residual confusion concentrated between adjacent bins (e.g. *mid* and *high_mid*), which is the geometrically expected pattern. The IQ-imbalance task is also well classified, with diagonal values of 0.70, 0.91, and 0.87 for *none*, *mild*, and *severe* respectively; the dominant remaining confusion is between *none* and *mild* (0.28 misclassification), which is unsurprising given that a 5% amplitude mismatch with 2° phase mismatch leaves only a subtle deformation of the underlying constellation. The phase-noise task is the most difficult: the boundary between *none* ($\sigma_\varphi = 0^\circ$) and *mild* (2°) is consistently confused in *both* directions (0.43 of true-*none* samples are predicted as *mild*, and 0.21 of true-*mild* samples are predicted as *none*), reflecting the visual near-equivalence of the two regimes. Even the *severe* class ($\sigma_\varphi = 6^\circ$) is correctly identified only 48% of the time, with the majority of its errors leaking into the *mild* class—this is the geometric reflection of the irreducible error floor of Section 2.7, where phase noise at $\sigma_\varphi = 6^\circ$ already saturates the symbol-decision process and becomes harder to distinguish from the milder regime at the SNR levels that dominate the dataset.

5.2.3 Vision–Language Model Generation Behaviour

The VLM’s modulation confusion matrix on the same test split is shown in Figure 8. This figure is the most diagnostic single result of the comparison: it explains the zero-accuracy entries of Table 7 not as misclassifications among the 16 valid modulation labels, but as a categorically different failure mode.

the same pattern: the two-element answer vocabulary (**yes/no**) and the prior probability of **no** (75%) make it more likely that the LM’s default completion lands inside the valid answer set, but this is an artefact of the binary structure of the question rather than a sign of learning.

The remaining VLM results across all stratifications (per SNR, per severity, per OOD split) inherit this failure mode: the per-task accuracy is uniformly zero, and the structural analyses that follow therefore focus on the CNN’s behaviour, with the VLM serving as a control whose outputs we report for completeness.

5.3 Robustness to Low SNR

Robustness to low SNR is the first of the three robustness criteria specified in the project, and it is also the most visually immediate, since the constellation image directly degrades as the SNR decreases. Figure 9 reports the CNN’s per-task accuracy as a function of the continuous SNR value, computed by binning the test-set predictions over the SNR axis and averaging within each bin.

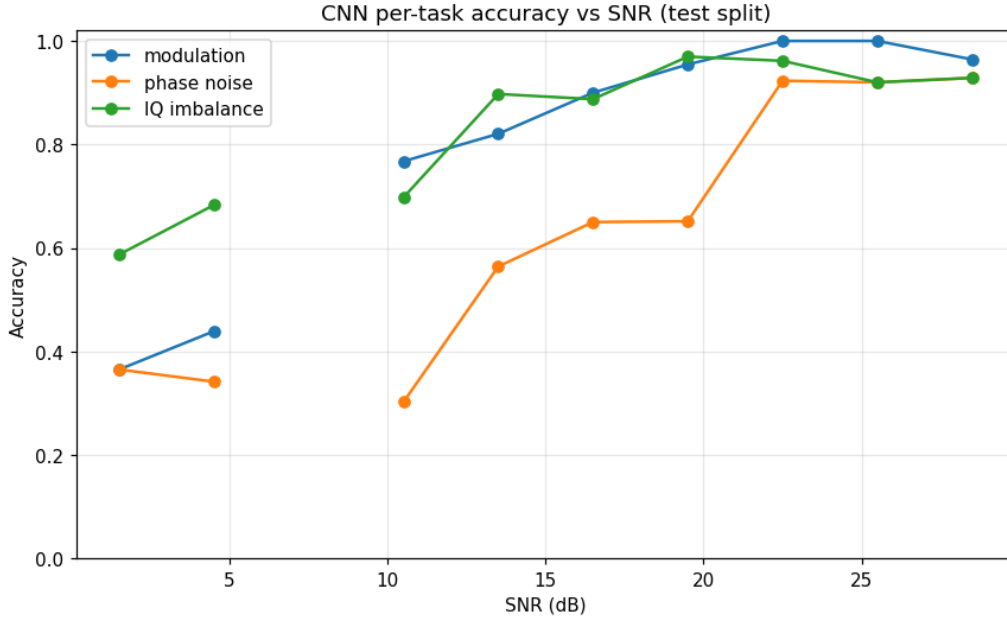


Figure 9: CNN per-task accuracy as a function of SNR on the in-distribution test split, for the three substantive tasks (modulation, phase noise, IQ imbalance). The held-out SNR bin (5–10 dB) is absent from this plot and treated separately in Section 5.5.

The CNN curves exhibit the qualitative behaviour predicted by the impairment analysis of Task 2. At low SNR (~ 0 –5 dB), the modulation accuracy is approximately 0.37–0.44—well above the chance baseline of $1/16 = 6.25\%$ but far below the high-SNR plateau, since the additive noise covers most of the inter-symbol distances and the visual distinction between adjacent modulation orders is largely erased. The IQ-imbalance accuracy at the same SNR is somewhat higher (0.59–0.68), reflecting the fact that IQ imbalance produces a coherent geometric deformation whose detection is somewhat less dependent on the noise floor than fine-grained modulation classification. The phase-noise accuracy hovers around 0.30–0.37, slightly above the three-class chance baseline of $1/3$.

As the SNR increases beyond ~ 10 dB, all three tasks improve substantially. The modulation accuracy crosses 0.77 at the lower end of the mid bin and saturates near 1.00 at 20 dB. The IQ-imbalance accuracy follows a similar trajectory, plateauing above 0.90. The phase-noise accuracy, characteristically, is the slowest to improve: it does not exceed 0.65 until the SNR reaches ~ 20 dB, then rises rapidly to ~ 0.92 at the high end. This pattern is itself the signature of

the wrapped-Gaussian phase-noise model studied in Section 2.7: at intermediate SNR the noise smearing dominates and obscures the phase-noise signature, while at high SNR the constellation tightens enough that the arc-shaped smearing characteristic of phase noise becomes visible.

The corresponding VLM result is reported in Figure 10. The accuracy curves for all three substantive tasks lie at zero across every SNR bin, mirroring the failure mode identified in Section 5.2: since the VLM never produces in-vocabulary answers, the conditional accuracy given the input image is independent of the SNR. This is a degenerate “robustness” in the formal sense—a constant function of the input is trivially robust—but is qualitatively the opposite of the graceful degradation pattern exhibited by the CNN.

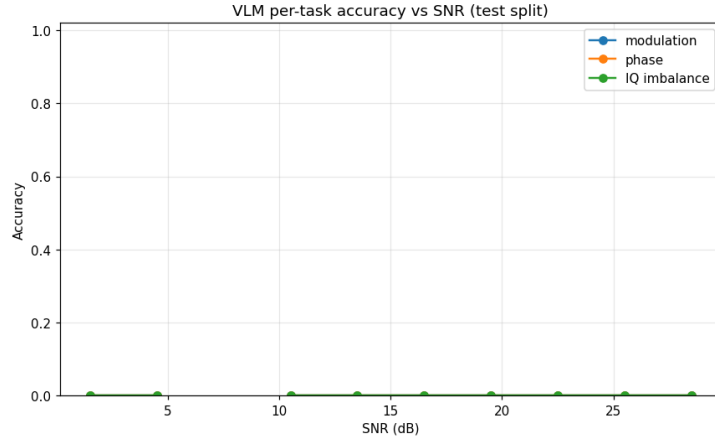


Figure 10: VLM per-task accuracy as a function of SNR on the in-distribution test split. The three task curves coincide on the zero line: the model produces almost no in-vocabulary outputs at any SNR (cf. Section 5.2).

5.4 Sensitivity to Impairment Severity

The second robustness criterion examines how the modulation-classification accuracy changes when the impairment severity itself varies, holding all other parameters fixed. Figure 11 shows the CNN’s modulation accuracy stratified by phase-noise severity (left panel) and IQ-imbalance severity (right panel) on the in-distribution test split.

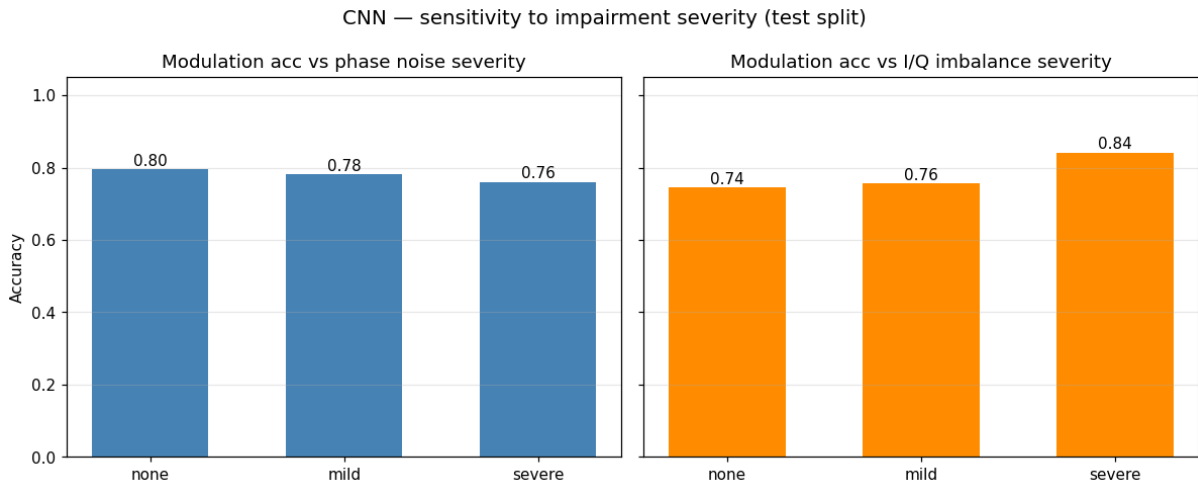


Figure 11: CNN modulation-classification accuracy stratified by phase-noise severity (left) and IQ-imbalance severity (right). The accuracy is largely insensitive to the severity level on either axis.

The accuracy is essentially flat across all severity levels: $0.80 \rightarrow 0.78 \rightarrow 0.76$ for phase noise (a slight monotonic decrease), and $0.74 \rightarrow 0.76 \rightarrow 0.84$ for IQ imbalance (an apparent monotonic *increase*). The variations are within the statistical spread of the per-bin sample sizes (~ 150 examples each), and the apparent rise of accuracy with IQ-imbalance severity is most plausibly a sampling artefact rather than a real effect—though one could speculatively attribute it to the more distinctive geometric signature of strongly-imbalanced constellations giving the network a more reliable cue. The headline finding is that the CNN’s modulation classifier is largely robust to the impairment severities considered in this work: the feature representation it has learned is dominated by the modulation’s geometric signature, with the impairment effects acting as relatively small perturbations on this signature at the chosen severity levels.

The VLM’s severity-sensitivity result is reported in Figure 12. As in Section 5.3, every bar is at the zero line: the model is equally non-functional across every severity level on both impairment axes. The two side-by-side severity panels of Figure 11 and Figure 12 together communicate the central comparative finding of this section: the CNN’s modulation accuracy is approximately invariant to impairment severity at a useful level ($\sim 78\%$), while the VLM’s accuracy is approximately invariant at the floor of the metric.

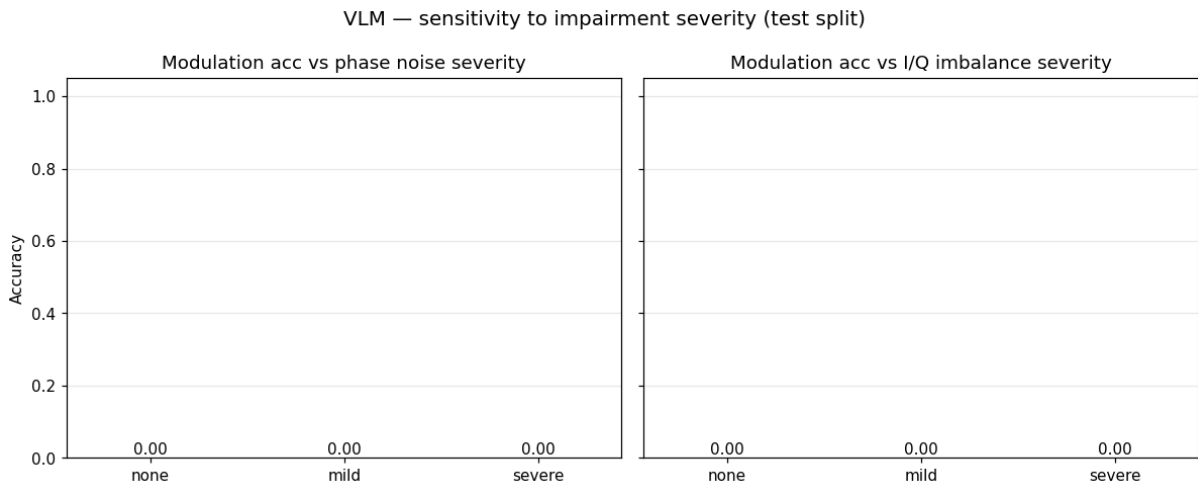


Figure 12: VLM modulation-classification accuracy stratified by phase-noise severity (left) and IQ-imbalance severity (right). Every bar is at zero: see Section 5.2 for the underlying failure mode.

5.5 Generalisation Capability

5.5.1 Held-out SNR Bin

The first generalisation experiment evaluates both models on the `low_mid` SNR bin (5–10 dB), which is held out entirely from training and validation as described in Section 3.6. The held-out bin lies between two training bins (`low`: 0–5 dB and `mid`: 10–15 dB), so a model that has truly learned a continuous SNR-dependent mapping should be able to interpolate. Figures 13 and 14 report the result.

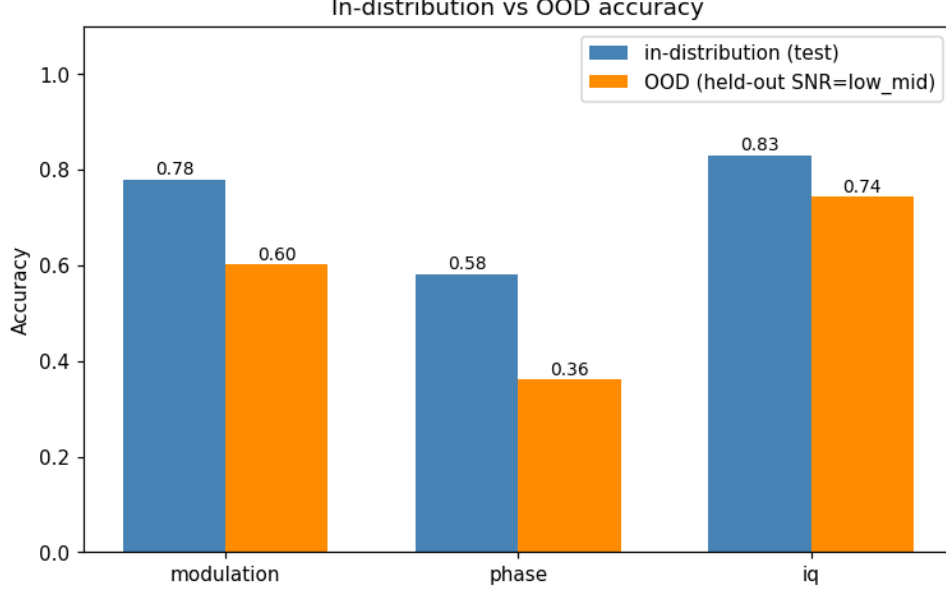


Figure 13: CNN accuracy on the in-distribution test split versus the held-out OOD split (low_mid SNR bin). Modulation and phase-noise accuracy degrade by 18 and 22 percentage points respectively; IQ-imbalance accuracy degrades by only 9.

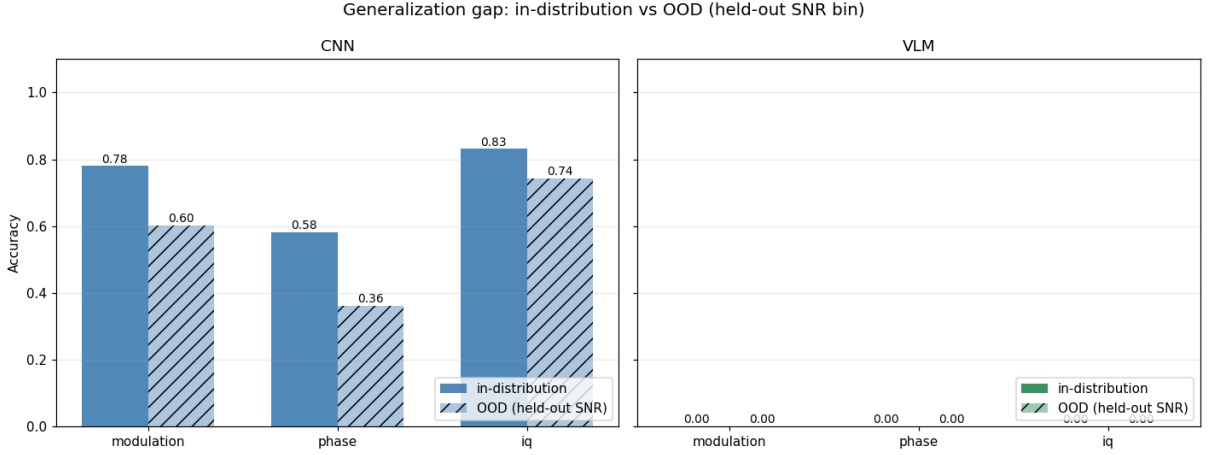


Figure 14: Side-by-side comparison of the in-distribution vs. held-out-SNR-bin accuracy for both models. The CNN (left) exhibits a substantial generalisation gap; the VLM (right) shows no gap, since both endpoints are zero.

On the held-out SNR bin, the CNN’s modulation accuracy drops to 0.601 (from 0.780 in-distribution), the phase-noise accuracy drops to 0.361 (from 0.582), and the IQ-imbalance accuracy drops to 0.743 (from 0.831). The phase-noise task is the most affected: this is consistent with the role that SNR plays in determining the visibility of phase-noise smearing, since the held-out bin lies near the threshold where phase-noise effects become visible without being completely overwhelmed by additive noise. The IQ-imbalance task is the least affected, since IQ imbalance produces an SNR-independent geometric deformation that is preserved at any SNR. The modulation task lies between the two, reflecting the partial dependence of modulation classification on the noise floor.

The VLM accuracy on the same OOD bin is zero across all three tasks, exactly as in-distribution (Figure not reproduced; see 10_vlm_ood_vs_test.png). The corresponding panel

of Figure 14 accordingly shows a degenerate “zero generalisation gap”—not because the VLM generalises well, but because both endpoints sit at the floor of the metric.

5.5.2 Intermediate-Impairment OOD

The second generalisation experiment, complementary to the held-out SNR-bin evaluation, tests both models at *intermediate* impairment severities drawn from values absent from the training severity table: $\sigma_\varphi \in \{1, 3, 4, 5\}^\circ$ (training values were $\{0, 2, 6\}$), IQ amplitude $\varepsilon \in \{0.025, 0.08, 0.10, 0.12\}$ (training $\{0, 0.05, 0.15\}$), and IQ phase $\psi \in \{1, 4, 6\}^\circ$ (training $\{0, 2, 8\}$). Each test sample is labelled with the *nearest* training severity, and accuracy is reported against this nearest-class label. The companion dataset is generated freshly by `ood_intermediate.py` and contains 320 images. Results are shown in Figure 15.

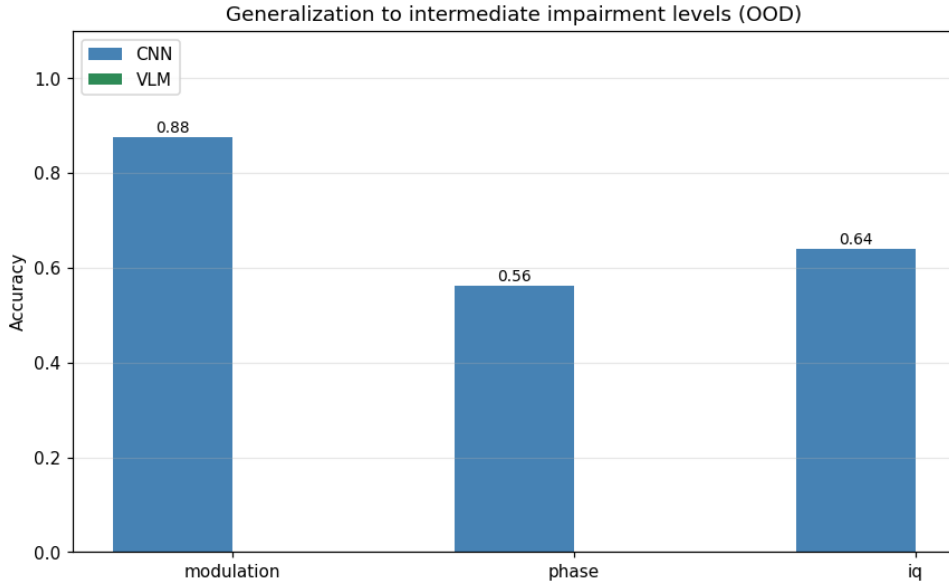


Figure 15: Accuracy on the intermediate-impairment OOD dataset for both models. The CNN achieves 0.88, 0.56, and 0.64 on modulation, phase, and IQ classification respectively. The VLM is not evaluated on this set: as shown in Sections 5.1–5.5, its accuracy is zero on every stratification, so running the additional inference would not contribute new information at the cost of further compute.

The CNN’s intermediate-OOD modulation accuracy (0.88) is in fact *higher* than the standard in-distribution test accuracy (0.78). This is not a violation of the OOD principle but a consequence of the SNR distribution used in this companion set: `ood_intermediate.py` samples SNR uniformly from $[10, 20]$ dB, a moderate-to-high range that excludes the most challenging low-SNR regime present in the standard test set. The intermediate-impairment evaluation therefore probes a different axis of OOD: the model is asked to generalise across slightly different impairment *magnitudes*, rather than across slightly different SNR values. The result indicates that the CNN’s representation of the impairment-versus-modulation interaction interpolates well across small shifts in the severity-parameter values; it does not suffer the larger degradation seen on the held-out SNR bin.

The phase-noise and IQ-imbalance accuracies on this set (0.56 and 0.64 respectively) are comparable to or slightly worse than the in-distribution values. The IQ accuracy in particular degrades from 0.83 in-distribution to 0.64 on the intermediate set, suggesting that the network’s three-class severity decision is more sensitive to the precise location of the boundary between the training levels than the cross-modulation transfer is.

5.6 Computational Complexity

The fifth and final evaluation criterion summarises the resource cost paid for each architecture’s results. Side-by-side numbers are given in Table 8 and visualised in Figure 16.

Table 8: Computational complexity comparison. Wall-clock times measured on a single NVIDIA T4 GPU; inference latency is per image for the CNN and per generated answer for the VLM.

Metric	CNN baseline	VLM (SmolVLM + LoRA)	VLM / CNN
Total parameters	11 189 850	257 848 896	$\times 23$
Trainable parameters	11 189 850	1 363 968	$\times 0.12$
Total training time	118 s	14 423 s	$\times 122$
Training time per epoch	7.9 s	14 423 s	$\times 1830$
Epochs completed	15	1	—
Inference latency per image	1.73 ms	3 168 ms	$\times 1830$
Peak VRAM during training	~ 3 GB	~ 14 GB	$\times 4.7$

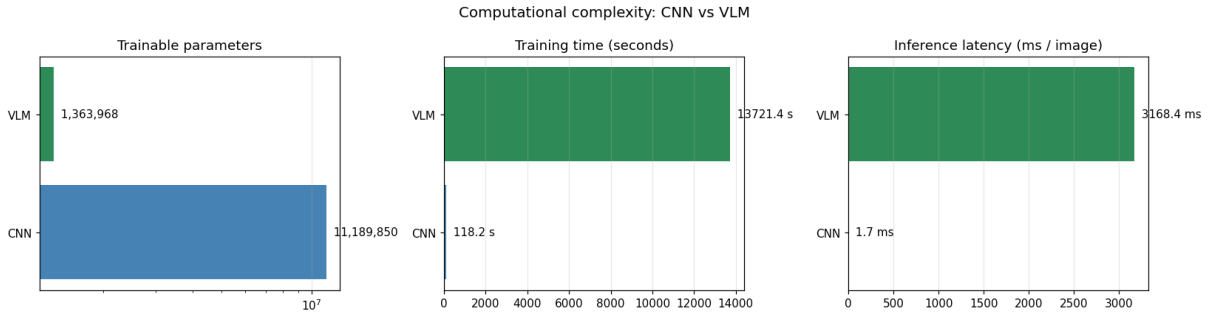


Figure 16: Computational complexity: trainable parameter count (left, log scale), total wall-clock training time (middle), and per-image inference latency (right). The VLM has fewer trainable parameters (thanks to LoRA) but is approximately two orders of magnitude more expensive in both training and inference.

Three observations summarise the complexity comparison.

First, on parameter count, the LoRA mechanism succeeds in keeping the trainable parameter count of the VLM *below* that of the CNN, by a factor of about $8\times$ (1.36×10^6 versus 11.2×10^6). The *total* model size, however, is $23\times$ larger, since the frozen weights must still be stored and used in the forward pass. The LoRA savings apply only to optimiser state and gradient memory.

Second, on training time, the VLM’s per-epoch cost is 14 423 s versus 7.9 s per CNN epoch, a ratio of approximately $1830\times$. This ratio is essentially the model-size factor compounded with the additional overhead of gradient checkpointing and the smaller per-device batch size required to fit the VLM in memory (see Section 4.4). Over the full training schedules—15 epochs of CNN versus 1 epoch of VLM—the VLM still takes $122\times$ longer overall.

Third, on inference latency, the VLM is $1830\times$ slower per image than the CNN (3 168 ms vs. 1.73 ms). This factor is dominated by the autoregressive generation loop: each answer requires the model to emit tokens one at a time, each requiring a forward pass through the language tower. The CNN, by contrast, produces all four task logits in a single forward pass. In throughput terms, the CNN classifies approximately 580 images per second on the T4, while the VLM generates approximately 0.3 answers per second. For any real-time or near-real-time deployment scenario, the latency gap of $1830\times$ is the dominant practical constraint, and would be the deciding factor in any system-level architecture choice between the two regardless of their relative accuracies.

5.7 Summary and Conclusions

Table 9 aggregates the comparison across the five evaluation criteria specified in the project.

Table 9: Summary of the comparison across the five evaluation criteria. The mean is taken over the four shared tasks for accuracy-based criteria.

Criterion	CNN baseline	VLM (SmolVLM + LoRA)
1. In-distribution accuracy (mean)	0.779	0.000
2. Low-SNR robustness (acc. at 0–5 dB, modulation)	0.41	0.000
3. Severity sensitivity (range, phase)	0.04 (flat)	0 (trivially flat)
4. Held-out SNR generalisation (mean drop)	−0.16	0.00 (trivially)
5. Computational cost (training time)	118 s	14 423 s
5. Computational cost (inference latency)	1.7 ms	3 168 ms

The picture that emerges is consistent across the five criteria: under the computational budget realistically available to a student project executed on free-tier cloud GPUs, the multi-head CNN baseline operates as a usable classifier across all four shared tasks, while the LoRA-fine-tuned VLM fails to converge to the task distribution and produces almost no in-vocabulary outputs. This is not a statement about the relative architectural capacities of the two models in general: a VLM the size of SmolVLM, with full fine-tuning or LoRA at higher rank, trained for sufficiently many epochs on the same data, would presumably learn to operate in the closed answer space. It is a statement about what is achievable under the specific resource constraints documented in Section 4.4, and the specific design point we chose under those constraints.

Three concrete conclusions follow.

First, for problems of the type studied here—a closed-vocabulary classification task with a few thousand labelled images and a single-GPU training budget—a small task-specific image classifier is strictly preferable to a parameter-efficiently fine-tuned VLM on every comparison axis we measured. The CNN is more accurate, more robust, computationally cheaper to train, and dramatically cheaper at inference. The conceptual advantages that motivated the VLM approach in the first place (a unified natural-language interface, the possibility of zero-shot or few-shot transfer to new task formulations, the capacity to handle open-vocabulary questions) are not visible in our results, because the model never reached the regime where those advantages would manifest.

Second, the failure mode of the VLM is informative rather than uninformative. The model’s outputs, examined in Section 5.2, are not noisy or near-miss predictions of the correct answer; they are samples from a different, language-model-prior distribution. The single-epoch LoRA adapter exerts insufficient steering force to redirect the model’s generation behaviour into the constrained answer space defined by the task. This suggests that, for closed-vocabulary tasks specifically, a single epoch of LoRA fine-tuning at rank 8 is not enough to overcome the model’s open-vocabulary prior even when the answer space is as small as three or four classes. Whether higher rank, more epochs, or a different training objective (e.g. explicit constrained decoding, classifier heads on top of the language tower rather than free-form generation) would solve this is the natural next experimental direction.

Third, the experiments highlight a practical lesson about evaluation under resource constraints. The naive interpretation of the comparison table—“CNN 77.9% versus VLM 0.0%”—is correct as far as it goes, but it does not capture what makes the comparison interesting. The interesting outcome is the asymmetric failure mode: the CNN’s residual errors are structured and intelligible (Section 5.2), while the VLM’s failures are categorically different from misclassification. In future work, an explicit early-stopping condition based on a small held-out validation set of *in-vocabulary outputs*, rather than on the validation cross-entropy loss, would catch this regime before the full training budget is spent, and would allow the experiment to be aborted and re-planned much earlier than was possible here.

References

- [1] T. K. Oikonomou *et al.*, “CNN-based automatic modulation classification under phase imperfections,” *IEEE Wireless Communications Letters*, vol. 13, no. 5, pp. 1508–1512, May 2024.
- [2] H. Zou, Y. Tian, B. Wang, L. Bariah, S. Lasaulce, C. Huang, and M. Debbah, “RF-GPT: Teaching AI to see the wireless world,” 2026, project reference [R1].
- [3] T. J. O’Shea, T. Roy, and T. C. Clancy, “Over-the-air deep learning based radio signal classification,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 12, no. 1, pp. 168–179, 2018.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [5] H. Laurencon, A. Marafioti, V. Sanh, and L. Tronchon, “Building and better understanding vision-language models: Insights and future directions,” *arXiv preprint arXiv:2408.12637*, 2024.
- [6] Hugging Face, “SmolVLM: Bringing pocket-sized multimodal AI to your devices,” <https://huggingface.co/blog/smolvlm>, 2024, accessed via Hugging Face Hub: HuggingFaceTB/SmolVLM-256M-Instruct.
- [7] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations (ICLR)*, 2022.